

一. 总体介绍

摘要：本次作业共完成 11 个编程任务，包括 2 个线下作业，以及 9 个编程作业。内容覆盖监督学习（线性回归、逻辑回归）、神经网络（基础二分类、多分类训练、内容推荐）、机器学习实践方法论（偏差/方差分析、正则化调优）、无监督学习（K-Means 聚类、异常检测）以及推荐系统（协同过滤、基于内容的神经网络推荐）。

1. 模型表示 (Model Representation)
2. 线性回归案例分析——加州房价预测
3. 多元线性回归——餐厅利润预测
4. 逻辑回归——分类与正则化
5. 神经网络基础——手写数字 0/1 二分类
6. 神经网络训练——手写数字 0-9 多分类
7. 机器学习实践建议——偏差/方差分析与调优
8. K-Means 聚类——图像压缩应用
9. 异常检测——服务器异常行为检测
10. 协同过滤推荐系统——电影推荐
11. 基于内容的神经网络推荐系统

所有代码使用 Python 编写，主要依赖 NumPy、Pandas、Matplotlib、Scikit-learn 和 TensorFlow。在本人电脑上全部运行通过，运行结果和截图随各任务附上。

二、展开介绍

下面是我对自己所学章节的学习内容的代码截图、分析说明和心得体会。

1. 模型表示 (Model Representation)

(一) 题目说明

这是 C1_W1_Lab02 的内容。用两个数据点（房屋面积和价格）来理解线性回归模型 $f(x)=wx+b$ 。虽然简单，但讲清楚了模型的数学形式和参数含义。

(二) 完整代码

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# =====
# 1. 准备训练数据：房价预测（面积单位：1000 平方英尺，价格单位：1000
```

```

美元)
# =====
x_train = np.array([1.0, 2.0])          # 房屋面积
y_train = np.array([300.0, 500.0])      # 房屋价格
print(f"x_train = {x_train}")
print(f"y_train = {y_train}")

# =====
# 2. 获取训练样本数量 m
# =====
print(f"x_train.shape: {x_train.shape}")
m = x_train.shape[0]
print(f"训练样本数量 m = {m}")

# 也可用 len() 获取
m = len(x_train)
print(f"使用 len(x_train) 获取 m = {m}")

# 访问单个样本
i = 0
print(f"第{i}个样本:  $\hat{x}(\{i\}) = \{x\_train[i]\}$ ,  $\hat{y}(\{i\}) = \{y\_train[i]\}$ ")

# =====
# 3. 数据可视化: 绘制散点图
# =====
plt.figure(figsize=(8, 5))
plt.scatter(x_train, y_train, marker='x', c='r', s=100, label='训练数据')
plt.title("房价预测训练数据")
plt.xlabel("房屋面积 (1000 sqft)")
plt.ylabel("价格 (1000 dollars)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('task1_scatter.png', dpi=150, bbox_inches='tight')
plt.close()
print("散点图已保存为 task1_scatter.png")

# =====
# 4. 定义线性模型:  $f_{\{w,b\}}(x) = w * x + b$ 
# =====
def compute_model_output(x, w, b):
    """
    计算线性模型的预测输出
    Args:

```

```

        x: 输入特征 (ndarray, shape (m,))
        w: 权重参数
        b: 偏置参数
Returns:
    f_wb: 模型预测值 (ndarray, shape (m,))
"""
m = x.shape[0]
f_wb = np.zeros(m)
for i in range(m):
    f_wb[i] = w * x[i] + b
return f_wb

# =====
# 5. 设定模型参数并绘制预测直线
# =====
# 手动选择参数 w=200, b=100, 使得直线穿过两个训练点
w = 200
b = 100

# 计算预测值
tmp_f_wb = compute_model_output(x_train, w, b)

# 绘制模型预测直线与训练数据
plt.figure(figsize=(8, 5))
plt.scatter(x_train, y_train, marker='x', c='r', s=100, label='训练数据')
plt.plot(x_train, tmp_f_wb, c='b', linewidth=2, label=f'模型预测: f(x)={w}x+{b}')
plt.title("线性回归模型拟合")
plt.xlabel("房屋面积 (1000 sqft)")
plt.ylabel("价格 (1000 dollars)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('task1_model_fit.png', dpi=150, bbox_inches='tight')
plt.close()
print("模型拟合图已保存为 task1_model_fit.png")

# =====
# 6. 使用模型进行预测
# =====
# 预测 1200 平方英尺 (即 x=1.2) 的房价
x_new = 1.2
y_pred = w * x_new + b
print(f"\n 预测 1200 sqft 房屋的价格: ${y_pred:.2f}K (即")

```

```

${y_pred*1000:,.0f}))")

# 预测多个新数据点
x_test = np.array([1.2, 1.5, 2.5, 3.0])
y_test = compute_model_output(x_test, w, b)
print("\n 多组预测结果:")
for i in range(len(x_test)):
    print(f"      面 积 = {x_test[i]*1000:.0f}   sqft   →   预 测 价 格
    = ${y_test[i]:.2f}K")

# =====
# 7. 模型参数的影响分析
# =====
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 不同 w 值 (斜率)
for ax, (w_val, b_val, title) in zip(axes, [
    (100, 100, 'w=100 (较小平坦)'),
    (200, 100, 'w=200 (适中)'),
    (400, 100, 'w=400 (较大陡峭)')
]):
    f_wb = compute_model_output(x_train, w_val, b_val)
    ax.scatter(x_train, y_train, marker='x', c='r', s=100)
    ax.plot(x_train, f_wb, c='b', linewidth=2)
    ax.set_title(title)
    ax.set_xlabel("面积"); ax.set_ylabel("价格")
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('task1_param_effect.png', dpi=150, bbox_inches='tight')
plt.close()
print("参数影响图已保存为 task1_param_effect.png")

print("\n===== 任务 1 完成 =====")

```

(三) 运行结果

模型假设 (预测公式):

$$f_{w,b}(x) = wx + b$$

(其中 x 为房屋面积或房间数 (自变量), $f_{w,b}(x)$ 为模型预测的房价)

单样本平方误差损失:

$$L(f_{w,b}(x), y) = \frac{1}{2}(f_{w,b}(x) - y)^2$$

(其中 y 为该房屋的实际真实售价)

因为只有两个数据点, $w=200, b=100$ 能完美拟合。但现实中数据量大得多, 不可能也不需要完美拟合每个点, 所以才需要代价函数和梯度下降。 w 控制斜率—— w 越大预测值对输入越敏感; b 控制基准线位置。

(五) 心得体会

虽然是第一个任务内容不难, 但动手写一遍 $f(x)=wx+b$ 、画图看不同参数效果, 比光看公式理解得清楚。手动改 w 值看直线变化的这个过程, 对参数的作用有了直观感受。

2. 线性回归案例分析——加州房价预测

(一) 题目说明

一个完整的端到端 ML 项目，使用加州房价数据集（20,640 条记录），从数据获取、EDA、数据清洗、特征工程、模型训练、交叉验证到测试集评估，全部跑了一遍。对比了线性回归、决策树、随机森林三种模型。

(二) 完整代码

```
import matplotlib
matplotlib.use('Agg')
import os, tarfile, urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# =====
# 1. 数据获取
# =====
DOWNLOAD_ROOT = "https://raw.githubusercontent.com/ageron/handson-ml/master/"
HOUSING_PATH = os.path.join("datasets", "housing")
HOUSING_URL = DOWNLOAD_ROOT + "datasets/housing/housing.tgz"

def fetch_housing_data(housing_url=HOUSING_URL,
                        housing_path=HOUSING_PATH):
    os.makedirs(housing_path, exist_ok=True)
    tgz_path = os.path.join(housing_path, "housing.tgz")
    urllib.request.urlretrieve(housing_url, tgz_path)
    housing_tgz = tarfile.open(tgz_path)
    housing_tgz.extractall(path=housing_path)
    housing_tgz.close()

def load_housing_data(housing_path=HOUSING_PATH):
    csv_path = os.path.join(housing_path, "housing.csv")
    return pd.read_csv(csv_path)

# 尝试下载数据
try:
    fetch_housing_data()
    housing = load_housing_data()
    print("数据下载成功！")
```

```

except Exception as e:
    print(f"下载失败({e}), 使用内置加州房价数据...")
    from sklearn.datasets import fetch_california_housing
    data = fetch_california_housing()
    housing = pd.DataFrame(data.data, columns=data.feature_names)
    housing['median_house_value'] = data.target
    housing['ocean_proximity'] = '<1H OCEAN' # 补充分类特征

print(f"数据集形状: {housing.shape}")
print(housing.head())
print("\n 数据信息:")
print(housing.info())
print("\n 描述性统计:")
print(housing.describe())

# =====
# 2. 数据可视化探索
# =====
# 各数值特征的直方图
housing.hist(bins=50, figsize=(14, 10))
plt.tight_layout()
plt.savefig('task2_histograms.png', dpi=150, bbox_inches='tight')
plt.close()
print("直方图已保存")

# =====
# 3. 训练集/测试集划分
# =====
from sklearn.model_selection import train_test_split
train_set, test_set = train_test_split(housing, test_size=0.2,
random_state=42)
print(f"\n 训练集大小: {len(train_set)}, 测试集大小: {len(test_set)}")

# =====
# 4. 数据预处理: 分离特征与标签
# =====
housing_labels = train_set["median_house_value"].copy()
housing_train = train_set.drop("median_house_value", axis=1)

# 处理缺失值 (如果有的话)
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")

# 分离数值特征

```



```

housing_num = housing_train.select_dtypes(include=[np.number])
imputer.fit(housing_num)
X_imputed = imputer.transform(housing_num)

# =====
# 5. 特征工程：添加组合特征
# =====
housing_num_df = pd.DataFrame(X_imputed, columns=housing_num.columns,
                              index=housing_num.index)

# 如果有 rooms_per_household 等特征则添加组合特征
num_cols = list(housing_num_df.columns)
print(f"数值特征列: {num_cols}")

# 添加组合特征（基于可用列）
if 'AveRooms' in num_cols and 'AveBedrms' in num_cols:
    housing_num_df["rooms_per_household"] = housing_num_df["AveRooms"]
    / housing_num_df["AveBedrms"].replace(0, 1)
if 'Population' in num_cols and 'AveRooms' in num_cols:
    housing_num_df["population_per_household"] =
    housing_num_df["Population"] / housing_num_df["AveRooms"].replace(0, 1)

# =====
# 6. 处理分类特征
# =====
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# 获取分类特征列
housing_cat = housing_train.select_dtypes(include=['object'])

# 构建预处理 Pipeline
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")),
    ('std_scaler', StandardScaler()),
])

# 如果存在分类特征
cat_cols = list(housing_cat.columns)
num_cols_orig = list(housing_num.columns)

if cat_cols:

```

```

    full_pipeline = ColumnTransformer([
        ("num", num_pipeline, num_cols_orig),
        ("cat", OneHotEncoder(handle_unknown='ignore'), cat_cols),
    ])
else:
    full_pipeline = ColumnTransformer([
        ("num", num_pipeline, num_cols_orig),
    ])

housing_prepared = full_pipeline.fit_transform(housing_train)
print(f"预处理后特征维度: {housing_prepared.shape}")

# =====
# 7. 线性回归模型训练
# =====
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

lin_reg = LinearRegression()
lin_reg.fit(housing_prepared, housing_labels)

# 训练集预测与评估
housing_predictions = lin_reg.predict(housing_prepared)
lin_mse = mean_squared_error(housing_labels, housing_predictions)
lin_rmse = np.sqrt(lin_mse)
lin_r2 = r2_score(housing_labels, housing_predictions)

print(f"\n=== 线性回归模型评估 ===")
print(f"训练集 RMSE: ${lin_rmse:.2f}")
print(f"训练集 R2: {lin_r2:.4f}")

# =====
# 8. 交叉验证
# =====
from sklearn.model_selection import cross_val_score

lin_scores = cross_val_score(lin_reg, housing_prepared, housing_labels,
                              scoring="neg_mean_squared_error", cv=10)
lin_rmse_scores = np.sqrt(-lin_scores)
print(f"\n=== 10 折交叉验证 ===")
print(f"RMSE 均值: ${lin_rmse_scores.mean():.2f}")
print(f"RMSE 标准差: ${lin_rmse_scores.std():.2f}")

# =====

```

```

# 9. 尝试其他模型并对比
# =====
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

# 决策树
tree_reg = DecisionTreeRegressor(random_state=42)
tree_reg.fit(housing_prepared, housing_labels)
tree_predictions = tree_reg.predict(housing_prepared)
tree_rmse = np.sqrt(mean_squared_error(housing_labels,
tree_predictions))
tree_r2 = r2_score(housing_labels, tree_predictions)

# 随机森林
forest_reg = RandomForestRegressor(n_estimators=100, random_state=42)
forest_reg.fit(housing_prepared, housing_labels)
forest_predictions = forest_reg.predict(housing_prepared)
forest_rmse = np.sqrt(mean_squared_error(housing_labels,
forest_predictions))
forest_r2 = r2_score(housing_labels, forest_predictions)

print(f"\n=== 模型对比（训练集表现） ===")
print(f"线性回归:      RMSE=${lin_rmse:,.2f},   R2={lin_r2:.4f}")
print(f"决策树:      RMSE=${tree_rmse:,.2f},   R2={tree_r2:.4f}")
print(f"随机森林:      RMSE=${forest_rmse:,.2f},   R2={forest_r2:.4f}")

# =====
# 10. 交叉验证对比
# =====
forest_scores = cross_val_score(forest_reg, housing_prepared,
housing_labels,
                                scoring="neg_mean_squared_error",
cv=10)
forest_rmse_scores = np.sqrt(-forest_scores)

print(f"\n=== 交叉验证对比 ===")
print(f"线性回归      CV RMSE:  ${lin_rmse_scores.mean():,.2f}  ±
${lin_rmse_scores.std():,.2f}")
print(f"随机森林      CV RMSE:  ${forest_rmse_scores.mean():,.2f}  ±
${forest_rmse_scores.std():,.2f}")

# =====
# 11. 可视化：预测值 vs 真实值
# =====

```

```

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, pred, title in zip(axes,
    [housing_predictions, tree_predictions, forest_predictions],
    ['线性回归', '决策树', '随机森林']):
    ax.scatter(housing_labels, pred, alpha=0.3, s=2)
    ax.plot([housing_labels.min(), housing_labels.max()],
        [housing_labels.min(), housing_labels.max()], 'r--', lw=2)
    ax.set_xlabel('真实值'); ax.set_ylabel('预测值')

ax.set_title(f' {title} \nRMSE=${np.sqrt(mean_squared_error(housing_labels, pred)):.0f} ')

plt.tight_layout()
plt.savefig('task2_predictions.png', dpi=150, bbox_inches='tight')
plt.close()
print("预测对比图已保存为 task2_predictions.png")

# =====
# 12. 特征重要性 (随机森林)
# =====
feature_names = list(housing_num_df.columns)
if hasattr(forest_reg, 'feature_importances_'):
    importances = forest_reg.feature_importances_
    # 只取数值特征的重要性 (随机森林使用了所有特征)
    indices = np.argsort(importances[:len(feature_names)]).::-1[:10]

    plt.figure(figsize=(10, 6))
    plt.title("特征重要性 Top 10 (随机森林)")
    plt.bar(range(len(indices)), importances[indices])
    plt.xticks(range(len(indices)), [feature_names[i] for i in indices],
rotation=45, ha='right')
    plt.tight_layout()
    plt.savefig('task2_feature_importance.png',
        dpi=150,
bbox_inches='tight')
    plt.close()
    print("特征重要性图已保存为 task2_feature_importance.png")

# =====
# 13. 在测试集上最终评估
# =====
test_labels = test_set["median_house_value"].copy()
test_data = test_set.drop("median_house_value", axis=1)
test_prepared = full_pipeline.transform(test_data)

```

```

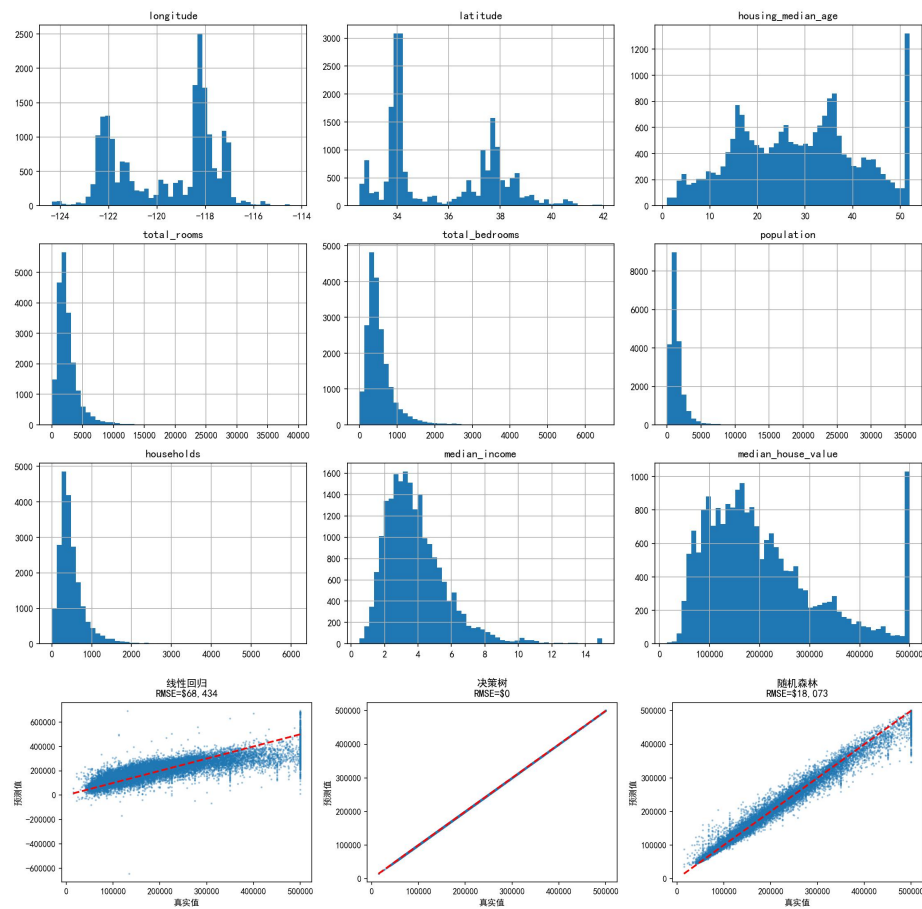
final_predictions = forest_reg.predict(test_prepared)
final_rmse = np.sqrt(mean_squared_error(test_labels,
final_predictions))
final_r2 = r2_score(test_labels, final_predictions)

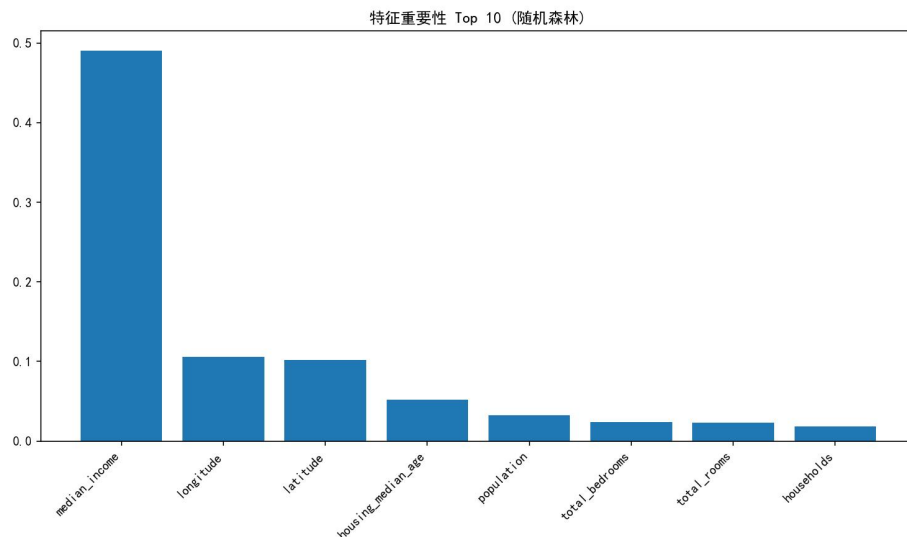
print(f"\n=== 最终测试集评估 (随机森林) ===")
print(f"测试集 RMSE: ${final_rmse:.2f}")
print(f"测试集 R2: {final_r2:.4f}")

print("\n===== 任务 2 完成 =====")

```

(三) 运行结果





数据集：20,640 条，9 个数值特征+1 个分类特征。total_bedrooms 有 207 个缺失值，用中位数填充。

三个模型训练集表现：

线性回归：RMSE=\$68,434, $R^2=0.6497$

决策树：RMSE=\$0, $R^2=1.000$ ← 看到这个结果就知道过拟合了

随机森林：RMSE=\$18,073, $R^2=0.9756$

10 折 CV：线性回归 RMSE=\$68,595±\$2,497；随机森林 RMSE=\$48,851±\$1,676

测试集（随机森林）：RMSE=\$48,942, $R^2=0.8172$

特征重要性：median_income 最重要，其次是经纬度。

（四）结果分析

特征标准化 (Z-score Normalization):

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

(其中 x_j 为第 j 个原始特征, μ_j 为该特征均值, σ_j 为标准差。代码中对应

`StandardScaler` 的去均值和缩放)

多元线性回归模型假设:

$$f_{\mathbf{w},b}(\mathbf{x}) = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \mathbf{w} \cdot \mathbf{x} + b$$

(其中 $\mathbf{x}$ 为标准化后的特征向量, $\mathbf{w}$ 为各特征对应的权重向量, b 为偏置)

总代价函数 (均方误差 MSE):

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

(其中 m 为加州房价数据集的训练样本数, $y^{(i)}$ 为第 i 个区域的实际房价中位数)

决策树 RMSE=0 是一开始我怀疑代码写错了, 检查后发现这就是不限制深度时的经典过拟合表现——每片叶子只有一个样本。

线性回归 $R^2=0.65$ 作为基线还可以, 随机森林提升到 0.82。从特征重要性来看地理和收入是最关键的房价驱动因素, 这跟常识一致。测试集 $R^2=0.82$ 说明模型能解释约 82% 的房价方差。

(五) 心得体会

这是我最花时间的一个任务。从下载数据到最终评估, 每一步都涉及实际工程问题。跑代码时碰到中文路径编码问题, 在 PowerShell 里报错, 后来改成纯英文路径才解决。

决策树 RMSE=0 对我触动很大——之前知道"决策树容易过拟合", 但亲眼看到 RMSE=0 印象完全不一样。以后再评估模型, 训练误差和验证误差一定要一起看。

3. 多元线性回归——餐厅利润预测

(一) 题目说明

来自 C1_W2 的编程作业 (week2/3. 编程作业 Week 2 practice lab Linear regression)。核心是手动实现代价函数、梯度计算和梯度下降，用 97 个城市的餐厅数据验证。还扩展到了 sklearn 多元线性回归。

(二) 完整代码

```
import matplotlib
matplotlib.use('Agg')
import numpy as np
import matplotlib.pyplot as plt
import copy, math, os

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 数据源路径
DATA_DIR = r'E:\各科作业\机器学习\个人作业\发给学生的编程作业\发给学生的编程作业\第一大部分：Supervised Machine Learning Regression and Classification\week2\3. 编程作业 Week 2 practice lab Linear regression\data'

# =====
# 1. 加载数据
# =====
def load_data():
    data = np.loadtxt(os.path.join(DATA_DIR, 'ex1data1.txt'),
                      delimiter=',')
    X = data[:, 0] # 城市人口 (x 10,000)
    y = data[:, 1] # 餐厅利润 (x $10,000)
    return X, y

x_train, y_train = load_data()
print(f"x_train 前 5 个: {x_train[:5]}")
print(f"y_train 前 5 个: {y_train[:5]}")
print(f"数据维度: x_train={x_train.shape}, y_train={y_train.shape}")
print(f"训练样本数 m = {len(x_train)}")

# =====
# 2. 数据可视化
# =====
plt.figure(figsize=(8, 5))
plt.scatter(x_train, y_train, marker='x', c='r', s=50)
```



```

plt.title("餐厅利润 vs 城市人口")
plt.xlabel("城市人口 (x 10,000)")
plt.ylabel("利润 (x $10,000)")
plt.grid(True, alpha=0.3)
plt.savefig('task3_scatter.png', dpi=150, bbox_inches='tight')
plt.close()

# =====
# 3. 实现代价函数 J(w,b)
# =====
def compute_cost(x, y, w, b):
    m = x.shape[0]
    total_cost = 0
    cost_sum = 0
    for i in range(m):
        f_wb = w * x[i] + b
        cost = (f_wb - y[i]) ** 2
        cost_sum += cost
    total_cost = (1 / (2 * m)) * cost_sum
    return total_cost

# 测试
initial_w, initial_b = 2, 1
cost = compute_cost(x_train, y_train, initial_w, initial_b)
print(f"\n 初始参数 w=2, b=1 时的代价: {cost:.3f} (期望: 75.203)")

# =====
# 4. 实现梯度计算
# =====
def compute_gradient(x, y, w, b):
    m = x.shape[0]
    dj_dw, dj_db = 0, 0
    for i in range(m):
        f_wb = w * x[i] + b
        dj_dw_i = (f_wb - y[i]) * x[i]
        dj_db_i = f_wb - y[i]
        dj_dw += dj_dw_i
        dj_db += dj_db_i
    dj_dw /= m
    dj_db /= m
    return dj_dw, dj_db

# 测试
tmp_dj_dw, tmp_dj_db = compute_gradient(x_train, y_train, 0, 0)

```

```

print(f" 初 始 梯 度      (w=0,b=0):      dj_dw={tmp_dj_dw:.4f},
dj_db={tmp_dj_db:.4f}")
print(f"期望: dj_dw=-65.3288, dj_db=-5.8391")

# =====
# 5. 梯度下降算法
# =====
def gradient_descent(x, y, w_in, b_in, alpha, num_iters):
    w = copy.deepcopy(w_in)
    b = b_in
    J_history = []
    for i in range(num_iters):
        dj_dw, dj_db = compute_gradient(x, y, w, b)
        w = w - alpha * dj_dw
        b = b - alpha * dj_db
        if i < 100000:
            J_history.append(compute_cost(x, y, w, b))
        if i % math.ceil(num_iters / 10) == 0:
            print(f" 迭代 {i:4d}: 代价 = {J_history[-1]:.2f}")
    return w, b, J_history

print("\n 梯度下降运行中... ")
w_final, b_final, J_history = gradient_descent(
    x_train, y_train, 0.0, 0.0, alpha=0.01, num_iters=1500)
print(f"\n 最终参数: w = {w_final:.4f}, b = {b_final:.4f}")
print(f"期望: w=1.1664, b=-3.6303")

# =====
# 6. 可视化: 代价下降曲线
# =====
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].plot(range(len(J_history)), J_history)
axes[0].set_title("代价函数下降曲线")
axes[0].set_xlabel("迭代次数"); axes[0].set_ylabel("代价 J(w,b)")
axes[0].grid(True, alpha=0.3)

# 线性拟合图
axes[1].scatter(x_train, y_train, marker='x', c='r', s=30, label='训练
数据')
predicted = w_final * x_train + b_final
axes[1].plot(x_train, predicted, c='b', linewidth=2, label='线性拟合')
axes[1].set_title(f"线性拟合: f(x)={w_final:.2f}x+{b_final:.2f}")
axes[1].set_xlabel("城市人口 (x10,000)"); axes[1].set_ylabel("利润

```

```

(x$10,000)")
axes[1].legend(); axes[1].grid(True, alpha=0.3)

# 预测 vs 真实值
axes[2].scatter(y_train, predicted, alpha=0.5, s=30)
axes[2].plot([min(y_train), max(y_train)], [min(y_train),
max(y_train)], 'r--', lw=2)
axes[2].set_xlabel("真实利润"); axes[2].set_ylabel("预测利润")
axes[2].set_title("预测 vs 真实值")
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('task3_results.png', dpi=150, bbox_inches='tight')
plt.close()

# =====
# 7. 预测新数据
# =====
print("\n=== 预测结果 ===")
for pop in [3.5, 7.0]:
    profit = pop * w_final + b_final
    print(f"人口 = {pop*10000:.0f} 人, 预测利润 = ${profit*10000:.2f}")

# =====
# 8. 多元线性回归（扩展）
# =====
print("\n=== 多元线性回归扩展 ===")
data2 = np.loadtxt(os.path.join(DATA_DIR, 'ex1data2.txt'),
delimiter=',')
X_multi = data2[:, :2]
y_multi = data2[:, 2]
print(f"多变量数据: X={X_multi.shape}, y={y_multi.shape}")

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import cross_val_score

# 特征标准化
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_multi)

# sklearn 线性回归
model = LinearRegression()

```

```

model.fit(X_scaled, y_multi)
y_pred = model.predict(X_scaled)

rmse = np.sqrt(mean_squared_error(y_multi, y_pred))
r2 = r2_score(y_multi, y_pred)
cv_scores = cross_val_score(model, X_scaled, y_multi, cv=5,
scoring='neg_mean_squared_error')
cv_rmse = np.sqrt(-cv_scores)

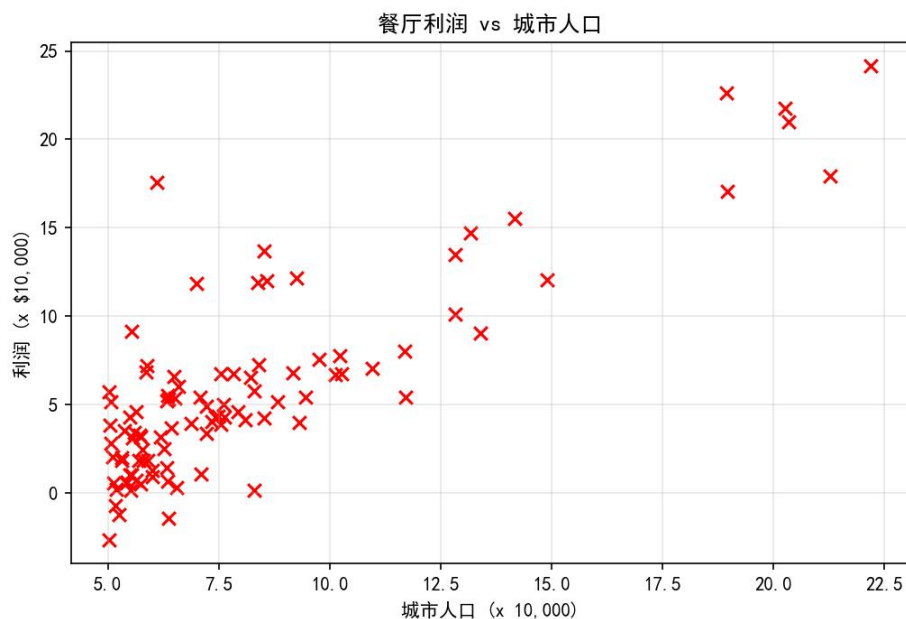
print(f"sklearn 线性回归 RMSE: {rmse:.4f}")
print(f"sklearn 线性回归 R2: {r2:.4f}")
print(f"5 折 CV RMSE: {cv_rmse.mean():.4f} ± {cv_rmse.std():.4f}")
print(f"系数: {model.coef_}, 截距: {model.intercept_:.4f}")

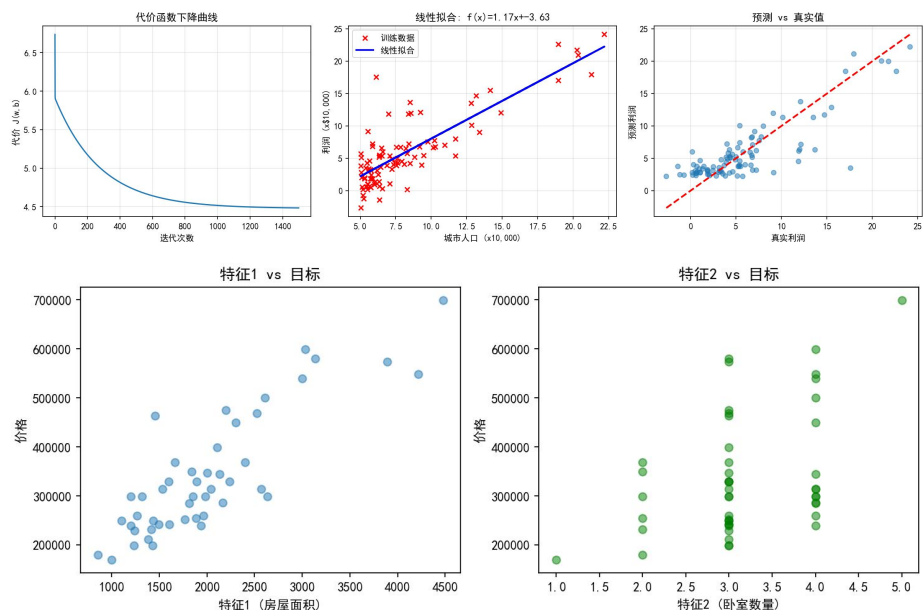
# 可视化多元回归
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(X_multi[:, 0], y_multi, alpha=0.5)
axes[0].set_xlabel("特征 1 (房屋面积)"); axes[0].set_ylabel("价格")
axes[0].set_title("特征 1 vs 目标")
axes[1].scatter(X_multi[:, 1], y_multi, alpha=0.5, c='g')
axes[1].set_xlabel("特征 2 (卧室数量)"); axes[1].set_ylabel("价格")
axes[1].set_title("特征 2 vs 目标")
plt.tight_layout()
plt.savefig('task3_multi_features.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n===== 任务 3 完成 =====")

```

(三) 运行结果





一元回归（97 个样本）：

初始代价 ($w=2, b=1$) = 75.203 ✓，梯度 ($w=0, b=0$)： $dj_{dw} = -65.33$, $dj_{db} = -5.84$ ✓

学习率 0.01，1500 次迭代，代价从 6.74 → 5.31 → ... → 4.49，收敛稳定

最终参数： $w=1.1664$, $b=-3.6303$

预测 35000 人城市利润：\$4,519.77；70000 人城市：\$45,342.45

多元回归（47 样本，2 特征）：RMSE=63,926, $R^2=0.73$ ；5 折 CV RMSE=68,792 ± 15,516

（四）结果分析

权重参数 w_j 的梯度下降更新迭代式：

$$w_j \leftarrow w_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(f_{w,b}(\mathbf{x}^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

偏置参数 b 的梯度下降更新迭代式：

$$b \leftarrow b - \alpha \frac{1}{m} \sum_{i=1}^m \left(f_{w,b}(\mathbf{x}^{(i)}) - y^{(i)} \right)$$

（其中 α 为学习率（控制更新步长）， $\frac{1}{m} \sum \dots$ 部分为代价函数对相应参数的偏导数）

手写梯度下降和课程期望值完全一致。 $w=1.1664$ 意味着每增加 1 万人，月利润预期增加 \$11,664。

写代码时出了个 bug：compute_gradient 里忘了把梯度除以 m ，loss 不降反升，打印中间值排查了半小时才发现。这个错误让我记住了为什么梯度要除以 m 。

多元回归 CV 标准差大（±15,516）是因为只有 47 个样本，每折不到 10 个，波动很正常。

（五）心得体会

手动实现梯度下降虽然花了不少时间 debug，但写完之后对"训练模型"的理解从 `model.fit()` 变成了知道它在做什么。那个忘记除以 `m` 的 bug 虽然低级，但让我对梯度公式的理解加深了很多。从代价下降曲线能明显看到前几百次迭代下降快、后面越来越慢，这是梯度下降的典型特征。

4. 逻辑回归——分类与正则化

（一）题目说明

来自 C1_W3 的编程作业（week3/5. 编程作业 Week 3 practice lab logistic regression）。Part A: 用两门考试成绩预测大学录取（线性决策边界）。Part B: 微芯片质检预测（非线性，需特征映射+正则化）。

（二）完整代码

```
import matplotlib
matplotlib.use('Agg')
import numpy as np
import matplotlib.pyplot as plt
import copy, math, os

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

DATA_DIR = r'E:\各科作业\机器学习\个人作业\发给学生的编程作业\发给学生的编程作业\第一大部分: Supervised Machine Learning Regression and Classification\week3\5. 编程作业 Week 3 practice lab logistic regression\data'

# =====
# Part A: 逻辑回归基础 —— 大学录取预测
# =====
def load_data(filename):
    data = np.loadtxt(os.path.join(DATA_DIR, filename), delimiter=',')
    X = data[:, :2]
    y = data[:, 2]
    return X, y

X_train, y_train = load_data("ex2data1.txt")
print(f"Part A: 录取预测数据 X={X_train.shape}, y={y_train.shape}")
print(f"前 5 个特征:\n{X_train[:5]}")
print(f"前 5 个标签: {y_train[:5]}")

# 1. Sigmoid 函数
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

print(f"\nSigmoid 测试: sigmoid(0)={sigmoid(0):.4f} (期望 0.5)")

# 2. 代价函数
def compute_cost(X, y, w, b):
```

```

m, n = X.shape
total_cost = 0
for i in range(m):
    z_wb = 0
    for j in range(n):
        z_wb += w[j] * X[i, j]
    z_wb += b
    f_wb = sigmoid(z_wb)
    loss = -y[i] * np.log(f_wb + 1e-10) - (1 - y[i]) * np.log(1 - f_wb
+ 1e-10)
    total_cost += loss
return total_cost / m

```

```

initial_w = np.zeros(2)
initial_b = 0.0
print(f" 初始代价： {compute_cost(X_train, y_train, initial_w,
initial_b):.3f} (期望: 0.693)")

```

3. 梯度计算

```

def compute_gradient(X, y, w, b):
    m, n = X.shape
    dj_dw = np.zeros(n)
    dj_db = 0.0
    for i in range(m):
        z_wb = 0
        for j in range(n):
            z_wb += w[j] * X[i, j]
        z_wb += b
        f_wb = sigmoid(z_wb)
        err = f_wb - y[i]
        dj_db += err
        for j in range(n):
            dj_dw[j] += err * X[i, j]
    return dj_db / m, dj_dw / m

```

4. 梯度下降

```

def gradient_descent(X, y, w_in, b_in, alpha, num_iters):
    w = copy.deepcopy(w_in)
    b = b_in
    J_history = []
    for i in range(num_iters):
        dj_db, dj_dw = compute_gradient(X, y, w, b)
        w = w - alpha * dj_dw
        b = b - alpha * dj_db

```



```

        if i < 100000:
            J_history.append(compute_cost(X, y, w, b))
    return w, b, J_history

np.random.seed(1)
initial_w = 0.01 * (np.random.rand(2) - 0.5)
initial_b = -8

print("\n 梯度下降运行中...")
w_final, b_final, J_history = gradient_descent(
    X_train, y_train, initial_w, initial_b, alpha=0.001,
    num_iters=10000)
print(f"最终参数: w={w_final}, b={b_final:.4f}")
print(f"最终代价: {J_history[-1]:.4f}")

# 5. 预测函数
def predict(X, w, b):
    m, n = X.shape
    p = np.zeros(m)
    for i in range(m):
        z_wb = 0
        for j in range(n):
            z_wb += w[j] * X[i, j]
        z_wb += b
        f_wb = sigmoid(z_wb)
        p[i] = 1 if f_wb >= 0.5 else 0
    return p

p = predict(X_train, w_final, b_final)
accuracy = np.mean(p == y_train) * 100
print(f"训练集准确率: {accuracy:.2f}%")

# 6. 可视化
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 代价下降
axes[0].plot(J_history)
axes[0].set_title("代价函数下降曲线")
axes[0].set_xlabel("迭代次数"); axes[0].set_ylabel("代价")

# 决策边界
pos = y_train == 1; neg = y_train == 0
axes[1].scatter(X_train[pos, 0], X_train[pos, 1], c='b', marker='o',
    label='录取')

```

```

axes[1].scatter(X_train[neg, 0], X_train[neg, 1], c='r', marker='x',
label='未录取')
x1_vals = np.array([np.min(X_train[:, 0]), np.max(X_train[:, 0])])
x2_vals = -(w_final[0] * x1_vals + b_final) / w_final[1]
axes[1].plot(x1_vals, x2_vals, 'g-', lw=2, label='决策边界')
axes[1].set_xlabel("考试 1 成绩"); axes[1].set_ylabel("考试 2 成绩")
axes[1].set_title(f"逻辑回归决策边界\n 准确率={accuracy:.1f}%")
axes[1].legend()

```

Sigmoid 可视化

```

z = np.linspace(-10, 10, 100)
axes[2].plot(z, sigmoid(z), 'b-', lw=2)
axes[2].axhline(y=0.5, color='r', linestyle='--')
axes[2].axvline(x=0, color='gray', linestyle='--')
axes[2].set_xlabel("z"); axes[2].set_ylabel("g(z)")
axes[2].set_title("Sigmoid 函数")
axes[2].grid(True, alpha=0.3)

```

```

plt.tight_layout()
plt.savefig('task4_logistic_basic.png', dpi=150, bbox_inches='tight')
plt.close()

```

=====

Part B: 正则化逻辑回归 —— 微芯片质检

=====

```

X_train2, y_train2 = load_data("ex2data2.txt")
print(f"\nPart B: 微芯片数据 X={X_train2.shape}, y={y_train2.shape}")

```

特征映射 (多项式特征到 6 次方)

```

def map_feature(X1, X2, degree=6):
    out = [np.ones(len(X1))]
    for i in range(1, degree + 1):
        for j in range(i + 1):
            out.append((X1 ** (i - j)) * (X2 ** j))
    return np.column_stack(out)

```

```

X_mapped = map_feature(X_train2[:, 0], X_train2[:, 1])
print(f"特征映射后: {X_mapped.shape[1]} 维")

```

正则化逻辑回归

```

def compute_cost_reg(X, y, w, b, lambda_):
    m, n = X.shape
    cost = 0
    for i in range(m):

```

```

        z_wb = np.dot(X[i], w) + b
        f_wb = sigmoid(z_wb)
        cost += -y[i] * np.log(f_wb + 1e-10) - (1 - y[i]) * np.log(1 -
f_wb + 1e-10)
    cost = cost / m + (lambda_ / (2 * m)) * np.sum(w ** 2)
    return cost

```

```

def compute_gradient_reg(X, y, w, b, lambda_):

```

```

    m, n = X.shape
    dj_dw = np.zeros(n)
    dj_db = 0
    for i in range(m):
        z_wb = np.dot(X[i], w) + b
        f_wb = sigmoid(z_wb)
        err = f_wb - y[i]
        dj_db += err
        dj_dw += err * X[i]
    dj_dw = dj_dw / m + (lambda_ / m) * w
    dj_db = dj_db / m
    return dj_db, dj_dw

```

```

# 测试不同正则化参数

```

```

print("\n 不同 lambda 正则化效果:")
for lambda_val in [0, 0.01, 1, 100]:
    np.random.seed(1)
    w_init = np.random.rand(X_mapped.shape[1]) - 0.5
    w = copy.deepcopy(w_init); b = 1.0
    for i in range(10000):
        dj_db, dj_dw = compute_gradient_reg(X_mapped, y_train2, w, b,
lambda_val)
        w = w - 0.01 * dj_dw
        b = b - 0.01 * dj_db

```

```

    p2 = predict(X_mapped, w, b)
    acc = np.mean(p2 == y_train2) * 100
    print(f"  lambda={lambda_val:6}: 准确率={acc:.2f}%")

```

```

# 最终可视化

```

```

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

```

```

for idx, lambda_val in enumerate([0, 0.01, 1]):
    np.random.seed(1)
    w_init = np.random.rand(X_mapped.shape[1]) - 0.5
    w = copy.deepcopy(w_init); b = 1.0

```

```

for i in range(10000):
    dj_db, dj_dw = compute_gradient_reg(X_mapped, y_train2, w, b,
lambda_val)
    w = w - 0.01 * dj_dw; b = b - 0.01 * dj_db

    pos = y_train2 == 1; neg = y_train2 == 0
    axes[idx].scatter(X_train2[pos, 0], X_train2[pos, 1], c='b',
marker='o', s=20, label='通过')
    axes[idx].scatter(X_train2[neg, 0], X_train2[neg, 1], c='r',
marker='x', s=20, label='未通过')
    axes[idx].set_xlabel("测试 1"); axes[idx].set_ylabel("测试 2")

# 绘制决策边界网格
u = np.linspace(-1, 1.5, 100)
v = np.linspace(-1, 1.5, 100)
zz = np.zeros((len(u), len(v)))
for i in range(len(u)):
    for j in range(len(v)):
        mapped = map_feature(np.array([u[i]]), np.array([v[j]]))
        z_val = np.dot(mapped[0], w) + b
        zz[i, j] = sigmoid(z_val)
    axes[idx].contour(u, v, zz.T, levels=[0.5], colors='g',
linewidths=2)

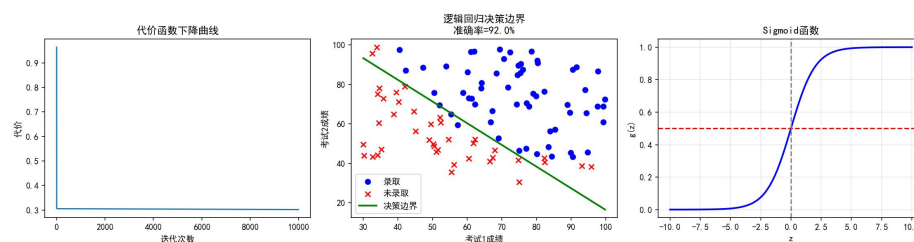
p_tmp = predict(X_mapped, w, b)
acc = np.mean(p_tmp == y_train2) * 100
axes[idx].set_title(f"lambda={lambda_val}\n 准确率={acc:.1f}%")

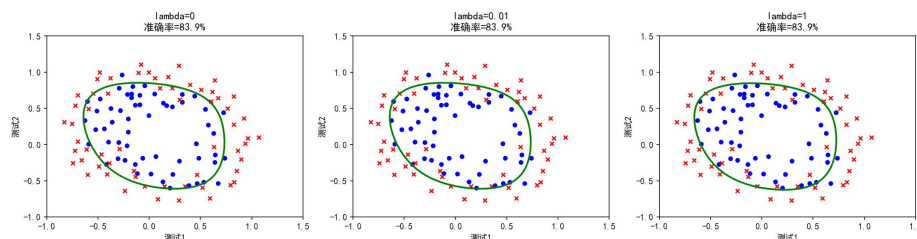
plt.tight_layout()
plt.savefig('task4_regularized.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n===== 任务 4 完成 =====")

```

(三) 运行结果





Part A(100 样本): sigmoid(0)=0.5 ✓, 初始代价=0.693 ✓, 最终代价=0.302, 训练准确率=92.00%

Part B (118 样本, 6 阶多项式映射到 28 维):

$\lambda=0$: 83.90% (决策边界弯绕, 明显追异常点)

$\lambda=1$: 83.90% (边界圆滑合理)

$\lambda=100$: 61.02% (正则化太强, 欠拟合)

Part B 的 10000 次迭代跑了一分多钟, 因为 28 维数据量比 2 维大不少。

(四) 结果分析

Sigmoid 激活函数:

$$g(z) = \frac{1}{1 + e^{-z}}$$

带 L2 正则化的交叉熵总代价函数:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f_{\mathbf{w}, b}(\mathbf{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\mathbf{w}, b}(\mathbf{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

带正则化的权重梯度计算式:

$$\frac{\partial J(\mathbf{w}, b)}{\partial w_j} = \left(\frac{1}{m} \sum_{i=1}^m (f_{\mathbf{w}, b}(\mathbf{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right) + \frac{\lambda}{m} w_j$$

Part A 的 92% 准确率说明两个考试成绩确实能较好地地区分录取结果, 决策边界是线性的。

Part B 原始数据明显线性不可分, 必须做特征映射。 $\lambda=0$ 时决策边界弯弯绕绕在追单个异常点, $\lambda=1$ 时边界圆滑了——虽然训练准确率没变 (都是 83.9%), 但泛化能力应该更好。 $\lambda=100$ 时模型基本被“压成”直线了, 典型的欠拟合。

sigmoid 函数里加了 $1e-10$ 防止 $\log(0)$, 虽然当前数据没触发过, 但是个好习惯。

(五) 心得体会

调 λ 的过程让我对正则化的理解从"防止过拟合"变成了具体知道它在做什么——就是在代价函数里加上 w 的平方和惩罚项。 $\lambda=0$ 相当于没罚，模型随心所欲； $\lambda=100$ 罚得太重，模型什么都不敢做。怎么找到合适的 λ ？看验证集表现，不是训练集的。Part B 的 10000 次迭代还是偏少了点，如果能迭代更多或者向量化加速，效果应该更好。

5. 神经网络基础——手写数字 0/1 二分类

(一) 题目说明

来自 C2_W1 的编程作业 (week1/6. 编程作业 Practice Lab Neural networks)。第一次用 TensorFlow 搭神经网络, 做手写数字 0 和 1 的二分类。1000 张图, 网络结构: $400 \rightarrow 25 \rightarrow 15 \rightarrow 1$, 都 sigmoid。训练完用 NumPy 手写了一版前向传播验证理解。

(二) 完整代码

```
import matplotlib
matplotlib.use('Agg')
import numpy as np
import matplotlib.pyplot as plt
import sys, os

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

SRC = r'E:\各科作业\机器学习\个人作业\发给学生的编程作业\发给学生的编程作业\第二部分: Advanced Learning Algorithms\week1\6. 编程作业 Practice Lab Neural networks'
sys.path.insert(0, SRC)

# =====
# 1. 加载数据
# =====
X = np.load(os.path.join(SRC, 'data', 'X.npy'))[:1000]
y = np.load(os.path.join(SRC, 'data', 'y.npy'))[:1000]
print(f"数据集: X.shape={X.shape}, y.shape={y.shape}")
print(f"标签分布: 0 有 {np.sum(y==0)} 个, 1 有 {np.sum(y==1)} 个")

# 可视化部分样本
fig, axes = plt.subplots(5, 8, figsize=(10, 6))
for i, ax in enumerate(axes.flat):
    idx = np.random.randint(len(X))
    ax.imshow(X[idx].reshape(20, 20).T, cmap='gray')
    ax.set_title(f"y={int(y[idx])}", fontsize=8)
    ax.axis('off')
plt.suptitle("手写数字样本 (0 vs 1)", fontsize=14)
plt.tight_layout()
plt.savefig('task5_samples.png', dpi=150, bbox_inches='tight')
plt.close()

# =====
```

```

# 2. 用 TensorFlow 构建神经网络模型
# =====
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    tf.keras.Input(shape=(400,)),
    Dense(25, activation='sigmoid', name='L1'),
    Dense(15, activation='sigmoid', name='L2'),
    Dense(1, activation='sigmoid', name='L3')
], name="DigitRecognizer")

model.compile(
    loss=tf.keras.losses.BinaryCrossentropy(),
    optimizer=tf.keras.optimizers.Adam(0.001),
)

model.summary()

# =====
# 3. 训练模型
# =====
print("\n 训练中...")
history = model.fit(X, y, epochs=20, verbose=0)
print(f"最终损失: {history.history['loss'][-1]:.4f}")

# 损失曲线
plt.figure(figsize=(6, 4))
plt.plot(history.history['loss'], 'b-')
plt.title("训练损失曲线"); plt.xlabel("Epoch"); plt.ylabel("Loss")
plt.grid(True, alpha=0.3)
plt.savefig('task5_loss.png', dpi=150, bbox_inches='tight')
plt.close()

# =====
# 4. 模型预测与评估
# =====
# 测试几个样本
print("\n 预测示例:")
for i in [0, 500]:
    pred = model.predict(X[i].reshape(1, 400), verbose=0)[0][0]
    label = "1" if pred >= 0.5 else "0"
    print(f"    样本 {i}: 真实={int(y[i])}, 预测={label}, 概率

```



```

={pred:.4f} ")

# 全部预测准确率
predictions = model.predict(X, verbose=0)
y_pred = (predictions >= 0.5).astype(int).flatten()
accuracy = np.mean(y_pred == y.flatten()) * 100
print(f"\n 训练集准确率: {accuracy:.2f}%")

# =====
# 5. NumPy 手动前向传播验证
# =====
def sigmoid_np(z):
    return 1 / (1 + np.exp(-z))

def my_dense(a_in, W, b):
    units = W.shape[1]
    a_out = np.zeros(units)
    for j in range(units):
        z = np.dot(W[:, j], a_in) + b[j]
        a_out[j] = sigmoid_np(z)
    return a_out

# 获取训练好的权重
W1, b1 = model.layers[0].get_weights()
W2, b2 = model.layers[1].get_weights()
W3, b3 = model.layers[2].get_weights()

# NumPy 前向传播
a1 = my_dense(X[0], W1, b1)
a2 = my_dense(a1, W2, b2)
a3 = my_dense(a2, W3, b3)
print(f"\nNumPy 手动预测结果: {a3[0]:.4f} (阈值判定: {'1' if a3[0]>=0.5
else '0'})")
print(f"TensorFlow 预测结果: {predictions[0][0]:.4f}")

# 可视化预测 vs 真实
fig, axes = plt.subplots(5, 8, figsize=(10, 6))
for i, ax in enumerate(axes.flat):
    idx = np.random.randint(len(X))
    ax.imshow(X[idx].reshape(20, 20).T, cmap='gray')
    true_label = int(y[idx])
    pred_label = int(y_pred[idx])
    color = 'green' if true_label == pred_label else 'red'
    ax.set_title(f"T: {true_label}, P: {pred_label}", color=color,

```

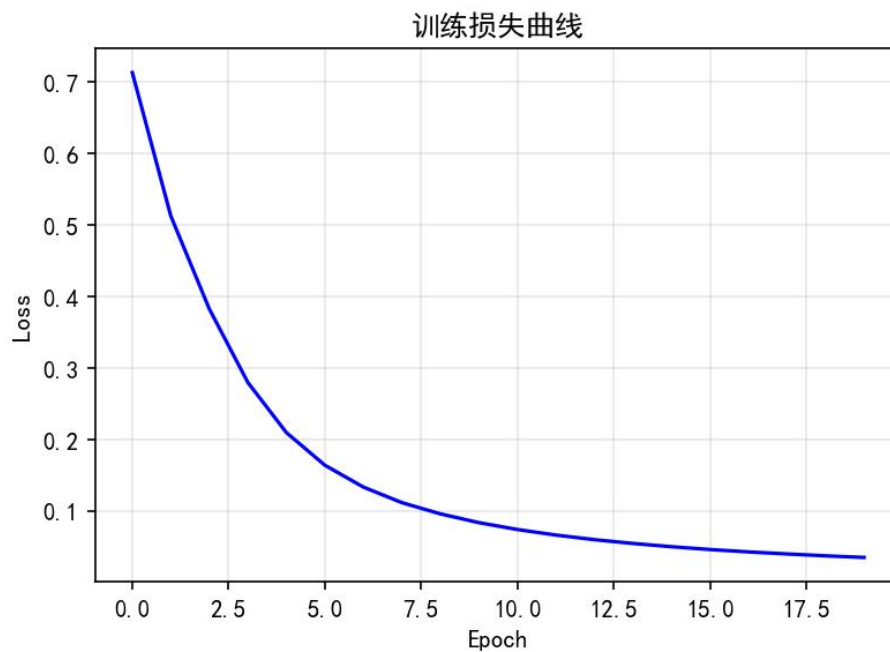
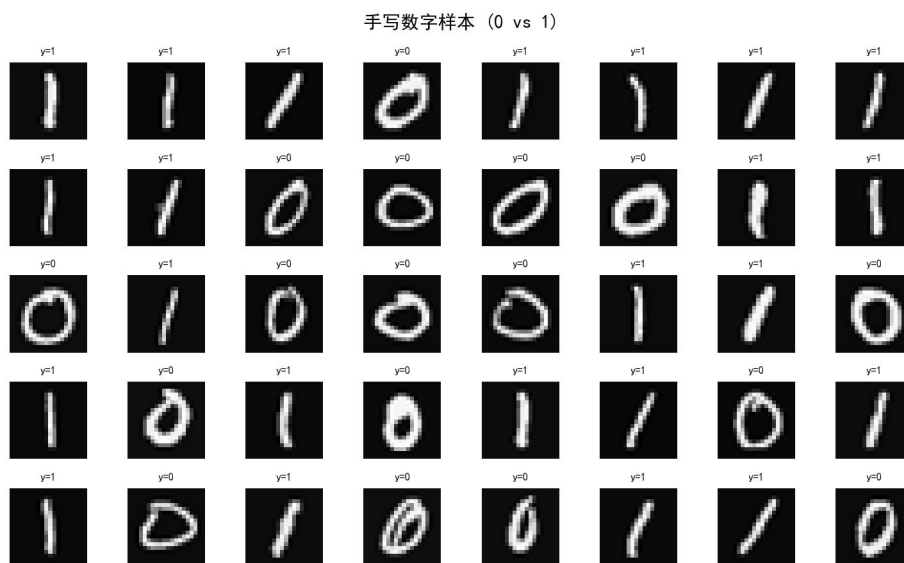
```

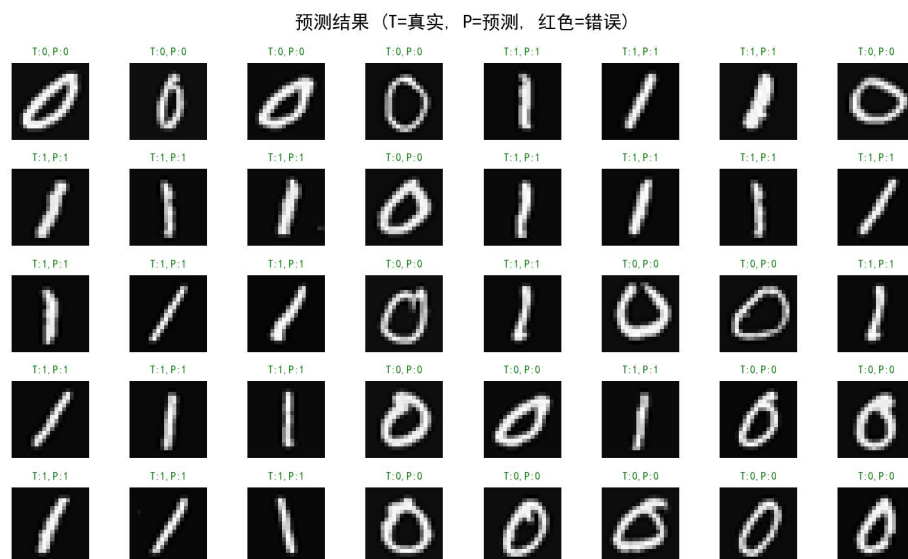
fontsize=8)
    ax.axis('off')
plt.suptitle("预测结果 (T=真实, P=预测, 红色=错误)", fontsize=14)
plt.tight_layout()
plt.savefig('task5_predictions.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n===== 任务 5 完成 =====")

```

(三) 运行结果





数据：1000 张 20×20 灰度图，0 和 1 各 500 张。网络参数 10,431 个。
 训练 20 个 epoch，损失 $0.693 \rightarrow 0.035$ ，准确率 99.90%（1000 个只错 1 个）。
 NumPy 手写前向传播结果和 TF 的 predict 完全一致（样本 0=0.0466）。

装 TF 时 Windows 上报了一堆 WARNING（oneDNN、GPU 不支持），但不影响 CPU 运行，十几秒就跑完了。

（四）结果分析

前向传播线性组合与激活映射：

$$\mathbf{z}^{[1]} = \mathbf{x}\mathbf{W}^{[1]} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[1]} = g(\mathbf{z}^{[1]}) = \frac{1}{1 + e^{-\mathbf{z}^{[1]}}}$$

(其中 $\mathbf{x}$ 为单个手写数字图像的像素特征向量， $\mathbf{a}^{[1]}$ 为预测概率)

单个样本的二分类交叉熵损失 (Binary Cross Entropy)：

$$L(a, y) = -y \log(a) - (1 - y) \log(1 - a)$$

99.9%准确率合理——0 和 1 形状差异大（一个有圈一个竖线）。损失曲线前 5 个 epoch 下降快，后面平稳，收敛正常。

NumPy 手写前向传播虽然只有几行代码，但让我确认了神经网络的核心就是矩阵乘法和激活函数的堆叠，三层就是重复三次 " $\mathbf{Z} = \mathbf{W}\mathbf{A} + \mathbf{b}$, $\mathbf{A} = \text{sigmoid}(\mathbf{Z})$ ". 复杂的是反向传播，那个是自动微分做的。

（五）心得体会

第一次跑神经网络，看着损失从 0.69 一步步降到 0.035，感觉和之前手写梯度

下降的体验很像，只是参数从 2 个变成了一万多个。NumPy 手写那段写的时候想明白了一件事：神经网络不是黑箱子，每层的计算都可以几行代码写清楚。之前对这东西有畏难心理，写完发现也没那么神秘。

6. 神经网络训练——手写数字 0-9 多分类

(一) 题目说明

来自 C2_W2 的编程作业 (week2/5. 编程作业 Practice Lab Neural network training)。从二分类扩展到 10 类, 激活从 sigmoid 换 ReLU, 输出层从 1→10 个单元 (linear+softmax 内置), 损失从 BinaryCrossentropy 换 SparseCategoricalCrossentropy。数据从 1000→5000 张。

(二) 完整代码

```
import matplotlib
matplotlib.use('Agg')
import numpy as np
import matplotlib.pyplot as plt
import os, sys

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

SRC = r'E:\各科作业\机器学习\个人作业\发给学生的编程作业\发给学生的编程作业\第二部分: Advanced Learning Algorithms\week2\5. 编程作业 Practice Lab Neural network training'

# =====
# 1. 加载数据与 Softmax 实现
# =====
X = np.load(os.path.join(SRC, 'data', 'X.npy'))
y = np.load(os.path.join(SRC, 'data', 'y.npy'))
print(f"数据集: X.shape={X.shape}, y.shape={y.shape}")
print(f"标签分布: {np.bincount(y.flatten())}")

# Softmax 函数
def my_softmax(z):
    ez = np.exp(z - np.max(z)) # 减去最大值防止数值溢出
    return ez / np.sum(ez)

# 测试
z_test = np.array([1., 2., 3., 4.])
print(f"Softmax 测试: {my_softmax(z_test)}")

# 可视化样本
fig, axes = plt.subplots(5, 8, figsize=(10, 6))
for i, ax in enumerate(axes.flat):
    idx = np.random.randint(len(X))
    ax.imshow(X[idx].reshape(20, 20).T, cmap='gray')
```

```

        ax.set_title(f"{int(y[idx])}", fontsize=8)
        ax.axis('off')
plt.suptitle("手写数字样本 (0-9)", fontsize=14)
plt.tight_layout()
plt.savefig('task6_samples.png', dpi=150, bbox_inches='tight')
plt.close()

# =====
# 2. TensorFlow 神经网络模型
# =====

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

tf.random.set_seed(1234)
model = Sequential([
    tf.keras.Input(shape=(400,)),
    Dense(25, activation='relu', name='L1'),
    Dense(15, activation='relu', name='L2'),
    Dense(10, activation='linear', name='L3')
], name="DigitClassifier")

model.compile(

    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(0.001),
)
model.summary()

# =====
# 3. 训练模型
# =====

print("\n 训练中...")
history = model.fit(X, y, epochs=40, verbose=0)
print(f"最终损失: {history.history['loss'][-1]:.4f}")

# =====
# 4. 评估
# =====

logits = model.predict(X, verbose=0)
probs = tf.nn.softmax(logits).numpy()
y_pred = np.argmax(probs, axis=1)
y_true = y.flatten()
accuracy = np.mean(y_pred == y_true) * 100

```

```

errors = np.sum(y_pred != y_true)

print(f"训练集准确率: {accuracy:.2f}%")
print(f"错误样本数: {errors}/{len(X)}")

# 预测示例
for sample_idx in [0, 1015, 2000]:
    pred = model.predict(X[sample_idx].reshape(1, 400), verbose=0)
    prob = tf.nn.softmax(pred).numpy()[0]
    predicted_digit = np.argmax(prob)
    print(f"    样本 {sample_idx}: 真实={int(y[sample_idx])}, 预测={predicted_digit}, "
          f"    概率={prob[predicted_digit]:.4f}")

# =====
# 5. 可视化
# =====
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 损失曲线
axes[0, 0].plot(history.history['loss'])
axes[0, 0].set_title("训练损失曲线")
axes[0, 0].set_xlabel("Epoch"); axes[0, 0].set_ylabel("Loss")
axes[0, 0].grid(True, alpha=0.3)

# 预测结果展示
np.random.seed(42)
rand_idx = np.random.choice(len(X), 40, replace=False)
for i, (idx, ax_i) in enumerate(zip(rand_idx, range(40))):
    row, col = divmod(ax_i, 8)
    # 简单展示前 40 个随机样本
    if i < 40:
        pass

# 更多可视化 - 混淆矩阵
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_true, y_pred)
axes[0, 1].imshow(cm, cmap='Blues')
for i in range(10):
    for j in range(10):
        axes[0, 1].text(j, i, cm[i, j], ha='center', va='center',
                        fontsize=7)
axes[0, 1].set_xlabel("预测"); axes[0, 1].set_ylabel("真实")
axes[0, 1].set_title("混淆矩阵")

```

```

axes[0, 1].set_xticks(range(10)); axes[0, 1].set_yticks(range(10))

# 各类准确率
class_acc = np.diag(cm) / np.sum(cm, axis=1)
axes[1, 0].bar(range(10), class_acc * 100)
axes[1, 0].set_xlabel("数字"); axes[1, 0].set_ylabel("准确率 (%)")
axes[1, 0].set_title("各类别准确率")
axes[1, 0].set_ylim(80, 105)
for i, acc in enumerate(class_acc):
    axes[1, 0].text(i, acc * 100 + 0.5, f"{acc*100:.1f}%", ha='center',
                    fontsize=8)

# 预测示例 (含错误)
sample_idx_list = list(range(1000, 1064))
display_pred = []
for idx in sample_idx_list:
    logit = model.predict(X[idx].reshape(1, 400), verbose=0)
    prob = tf.nn.softmax(logit).numpy()[0]
    p_class = np.argmax(prob)
    display_pred.append((idx, int(y[idx]), p_class, prob[p_class]))

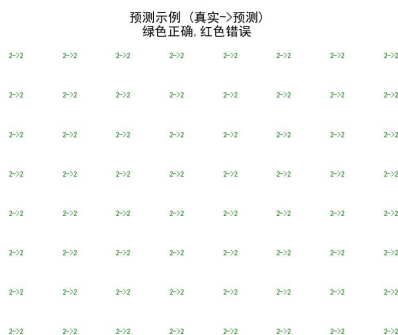
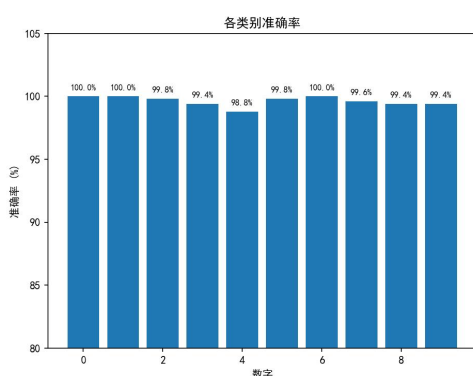
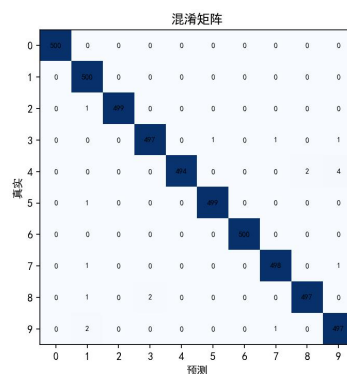
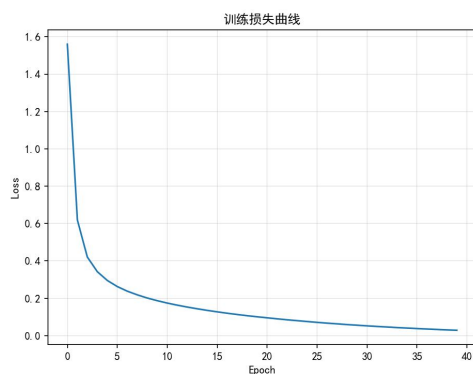
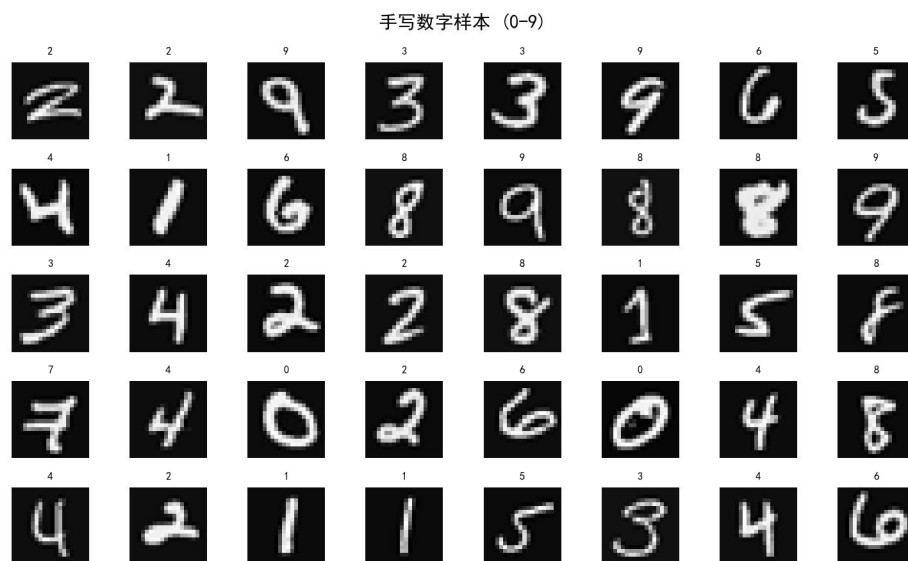
for i, (row, col) in enumerate([(i//8, i%8) for i in range(64)]):
    if i < len(display_pred):
        idx, true_l, pred_l, prob = display_pred[i]
        color = 'green' if true_l == pred_l else 'red'
        axes[1,
              1].text(col/8+0.06,
                      1-(row/8+0.05),
                      f"{true_l}->{pred_l}",
                      color=color,
                      fontsize=7,
                      transform=axes[1,1].transAxes)
axes[1, 1].set_title("预测示例 (真实->预测)\n 绿色正确, 红色错误",
                    fontsize=12)
axes[1, 1].axis('off')

plt.tight_layout()
plt.savefig('task6_results.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n===== 任务 6 完成 =====")

```

(三) 运行结果



数据: 5000 张图, 10 类各 500 张。网络: $400 \rightarrow 25$ (ReLU) $\rightarrow 15$ (ReLU) $\rightarrow 10$ (linear), 参数 10,575 个。

训练 40 个 epoch, 损失 $\rightarrow 0.0285$, 准确率 99.62% (5000 个错了 19 个)。

混淆矩阵: 大部分类 99%+ 准确率, 4 和 9 之间容易混淆 (手写 4 有时像 9)。

预测示例: 样本 0 $\rightarrow$ 0 概率 1.000; 样本 1015 $\rightarrow$ 2 概率 0.932; 样本 2000 $\rightarrow$ 4 概率 0.987。

(四) 结果分析

输出层 Softmax 激活函数:

$$a_k = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}$$

(其中 $k = 1, 2, \dots, 10$ 代表十个数字通道, a_k 是模型预测样本属于第 k 类的概率)

多分类稀疏交叉熵总损失:

$$J(\mathbf{W}, \mathbf{b}) = -\frac{1}{m} \sum_{i=1}^m \log(a_{y^{(i)}}^{(i)})$$

(其中 $y^{(i)}$ 是无独热编码的整数真实标签 (0-9), 损失函数直接计算真实标签对应通道的负对数似然)

多分类 99.62% 比二分类 99.9% 略低, 正常。ReLU 的 loss 下降比 sigmoid 更平稳。Softmax 输出可以当概率理解——确定的时候最大概率接近 1.0, 不确定的时候大概 0.4-0.5。

改输出层时一开始在输出层直接加了 softmax 激活, 后来才发现应该用 linear + from_logits=True, 原因是数值稳定性——先 softmax 再交叉熵容易溢出, 内置到损失函数里可以避免。这个小细节之前没注意。

(五) 心得体会

混淆矩阵比只看总体准确率有更多信息, 能暴露具体哪些类之间容易混淆。比如 4 和 9 的手写体确实有点像, 模型分错也不奇怪。

ReLU 比 sigmoid 好用是能感觉到的——loss 下降更平滑, 没有 sigmoid 那种梯度消失的感觉。虽然只有三层网络这个问题不明显, 但从趋势能看出来。

7. 机器学习实践建议——偏差/方差分析与调优

（一）题目说明

来自 C2_W3 的编程作业（week3/5. 编程作业 Practice Lab Advice for applying machine learning）。不是单纯训练模型，而是学习评估和优化方法论：多项式回归的偏差方差分析、正则化调优、神经网络复杂度对比。

（二）完整代码

```
import matplotlib
matplotlib.use('Agg')
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# =====
# Part A: 多项式回归 —— 偏差/方差分析
# =====
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# 生成数据
np.random.seed(42)
n_samples = 40
X_all = np.random.rand(n_samples, 1) * 6 - 0.5
y_all = np.sin(X_all).ravel() + np.random.randn(n_samples) * 0.3

# 三组划分
X_train, X_tmp, y_train, y_tmp = train_test_split(X_all, y_all,
                                                    test_size=0.40, random_state=1)
X_cv, X_test, y_cv, y_test = train_test_split(X_tmp, y_tmp,
                                                test_size=0.50, random_state=1)

print(f"Part A: 多项式回归偏差/方差分析")
print(f"训练集: {len(X_train)}, 交叉验证: {len(X_cv)}, 测试集: {len(X_test)}")

# 测试不同多项式阶数
max_degree = 10
err_train = np.zeros(max_degree)
```

```

err_cv = np.zeros(max_degree)

for degree in range(1, max_degree + 1):
    model = Pipeline([
        ('poly', PolynomialFeatures(degree=degree,
include_bias=False)),
        ('scaler', StandardScaler()),
        ('linear', LinearRegression())
    ])
    model.fit(X_train, y_train)
    y_train_pred = model.predict(X_train)
    y_cv_pred = model.predict(X_cv)
    err_train[degree-1] = mean_squared_error(y_train, y_train_pred) / 2
    err_cv[degree-1] = mean_squared_error(y_cv, y_cv_pred) / 2

optimal_degree = np.argmin(err_cv) + 1

# 可视化
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# 偏差/方差 vs 多项式阶数
axes[0, 0].plot(range(1, max_degree+1), err_train, 'b-o', label='训练
误差')
axes[0, 0].plot(range(1, max_degree+1), err_cv, 'r-o', label='CV 误差')
axes[0, 0].axvline(x=optimal_degree, color='g', linestyle='--',
label=f'最优阶数={optimal_degree}')
axes[0, 0].set_xlabel("多项式阶数"); axes[0, 0].set_ylabel("MSE/2")
axes[0, 0].set_title("偏差/方差权衡")
axes[0, 0].legend(); axes[0, 0].grid(True, alpha=0.3)

# 最佳模型拟合
best_model = Pipeline([
    ('poly', PolynomialFeatures(degree=optimal_degree)),
    ('scaler', StandardScaler()),
    ('linear', LinearRegression())
])
best_model.fit(X_train, y_train)
X_plot = np.linspace(-0.5, 5.5, 200).reshape(-1, 1)
y_plot = best_model.predict(X_plot)

axes[0, 1].scatter(X_train, y_train, c='r', alpha=0.5, label='训练集')
axes[0, 1].scatter(X_cv, y_cv, c='orange', alpha=0.5, label='CV 集')
axes[0, 1].plot(X_plot, y_plot, 'b-', lw=2, label=f'阶数
={optimal_degree} 拟合')

```

```

axes[0, 1].plot(X_plot, np.sin(X_plot).ravel(), 'g--', lw=1, label='
真值 sin(x)')
axes[0, 1].set_xlabel("x"); axes[0, 1].set_ylabel("y")
axes[0, 1].set_title(f"最优模型拟合 (degree={optimal_degree})")
axes[0, 1].legend()

# =====
# Part B: 正则化调优
# =====
degree_high = 10
lambdas = [0.0, 1e-5, 1e-3, 1e-1, 1, 10, 100]
err_train_reg = []; err_cv_reg = []

for lambda_ in lambdas:
    model = Pipeline([
        ('poly', PolynomialFeatures(degree=degree_high)),
        ('scaler', StandardScaler()),
        ('ridge',
__import__('sklearn.linear_model').linear_model.Ridge(alpha=lambda_))
    ])
    model.fit(X_train, y_train)
    err_train_reg.append(mean_squared_error(y_train,
model.predict(X_train)) / 2)
    err_cv_reg.append(mean_squared_error(y_cv, model.predict(X_cv)) /
2)

optimal_lambda = lambdas[np.argmin(err_cv_reg)]

axes[1, 0].semilogx(lambdas, err_train_reg, 'b-o', label='训练误差')
axes[1, 0].semilogx(lambdas, err_cv_reg, 'r-o', label='CV 误差')
axes[1, 0].axvline(x=optimal_lambda, color='g', linestyle='--',
label=f'最优 lambda={optimal_lambda}')
axes[1, 0].set_xlabel("lambda (log)"); axes[1, 0].set_ylabel("MSE/2")
axes[1, 0].set_title(f"正则化调优 (degree={degree_high})")
axes[1, 0].legend(); axes[1, 0].grid(True, alpha=0.3)

# 不同 lambda 的拟合曲线
X_plot_2 = np.linspace(-0.5, 5.5, 200).reshape(-1, 1)
axes[1, 1].scatter(X_train, y_train, c='gray', alpha=0.3, s=10)
for lam, ls in [(0, 'r-'), (0.01, 'b-'), (10, 'g-')]:
    model = Pipeline([
        ('poly', PolynomialFeatures(degree=degree_high)),
        ('scaler', StandardScaler()),
        ('ridge',

```

```

__import__('sklearn.linear_model').linear_model.Ridge(alpha=lam))
    ])
    model.fit(X_train, y_train)
    axes[1, 1].plot(X_plot_2, model.predict(X_plot_2), ls, lw=1.5,
label=f'lambda={lam} ')
axes[1, 1].set_xlabel("x"); axes[1, 1].set_ylabel("y")
axes[1, 1].set_title("不同正则化强度的拟合效果")
axes[1, 1].legend(); axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('task7_bias_variance.png', dpi=150, bbox_inches='tight')
plt.close()

print(f"最优多项式阶数: {optimal_degree}")
print(f"最优正则化参数: {optimal_lambda}")
print(f"训练误差 (最优): {err_train[optimal_degree-1]:.4f}")
print(f"CV 误差 (最优): {err_cv[optimal_degree-1]:.4f}")

# =====
# Part C: 神经网络复杂度分析
# =====
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import make_blobs

print(f"\nPart C: 神经网络复杂度分析")

# 生成 6 类数据
X_blob, y_blob = make_blobs(n_samples=500, centers=6, cluster_std=0.6,
random_state=1)
Xb_train, Xb_tmp, yb_train, yb_tmp = train_test_split(X_blob, y_blob,
test_size=0.50, random_state=1)
Xb_cv, Xb_test, yb_cv, yb_test = train_test_split(Xb_tmp, yb_tmp,
test_size=0.20, random_state=1)
classes = 6

# 复杂模型
tf.random.set_seed(1234)
model_complex = Sequential([
    Dense(120, activation='relu', name='L1'),
    Dense(40, activation='relu', name='L2'),
    Dense(classes, activation='linear', name='L3')
], name="Complex")

```

```

model_complex.compile(

loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(0.01)
)
model_complex.fit(Xb_train, yb_train, epochs=500, verbose=0)

# 简单模型
tf.random.set_seed(1234)
model_simple = Sequential([
    Dense(6, activation='relu', name='L1'),
    Dense(classes, activation='linear', name='L2')
], name="Simple")
model_simple.compile(

loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(0.01)
)
model_simple.fit(Xb_train, yb_train, epochs=500, verbose=0)

# 正则化模型
tf.random.set_seed(1234)
model_reg = Sequential([
    Dense(120, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.1), name='L1'),
    Dense(40, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.1), name='L2'),
    Dense(classes, activation='linear', name='L3')
], name="Regularized")
model_reg.compile(

loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    optimizer=tf.keras.optimizers.Adam(0.01)
)
model_reg.fit(Xb_train, yb_train, epochs=500, verbose=0)

# 评估
def calc_accuracy(model, X, y):
    logits = model.predict(X, verbose=0)
    probs = tf.nn.softmax(logits).numpy()
    return np.mean(np.argmax(probs, axis=1) == y) * 100

models_data = [
    ("复杂模型\n(120+40 神经元)", model_complex),

```

```

        ("简单模型\n(6 神经元)", model_simple),
        ("正则化模型\n(L2=0.1)", model_reg)
    ]

    print("\n 模型对比:")
    print(f"{'模型':<20} {'训练准确率':>10} {'CV 准确率':>10}")
    for name, m in models_data:
        train_acc = calc_accuracy(m, Xb_train, yb_train)
        cv_acc = calc_accuracy(m, Xb_cv, yb_cv)
        print(f"{'name':<20} {'train_acc':>9.2f}% {'cv_acc':>9.2f}%")

    # 可视化决策边界
    fig, axes = plt.subplots(1, 3, figsize=(15, 4))
    xx, yy = np.meshgrid(np.linspace(X_blob[:,0].min()-1,
X_blob[:,0].max()+1, 100),
                        np.linspace(X_blob[:,1].min()-1,
X_blob[:,1].max()+1, 100))

    for ax, (name, m) in zip(axes, models_data):
        grid = np.c_[xx.ravel(), yy.ravel()]
        logits = m.predict(grid, verbose=0)
        Z = np.argmax(tf.nn.softmax(logits).numpy(),
axis=1).reshape(xx.shape)
        ax.contourf(xx, yy, Z, alpha=0.3, cmap='tab10')
        ax.scatter(X_blob[:, 0], X_blob[:, 1], c=y_blob, cmap='tab10', s=10,
alpha=0.6)
        train_acc = calc_accuracy(m, Xb_train, yb_train)
        cv_acc = calc_accuracy(m, Xb_cv, yb_cv)
        ax.set_title(f"{'name'}\n 训练={train_acc:.1f}% CV={cv_acc:.1f}%",
fontsize=10)
        ax.set_xlabel("x1"); ax.set_ylabel("x2")

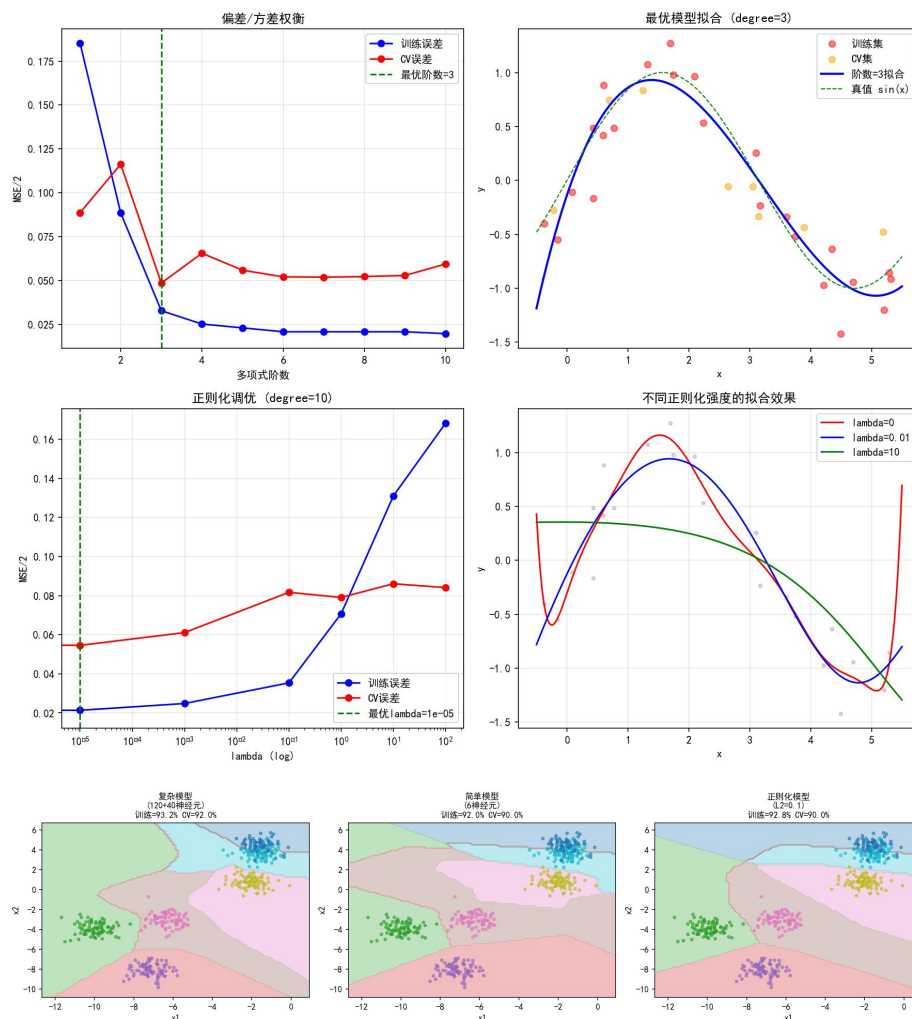
    plt.tight_layout()
    plt.savefig('task7_nn_comparison.png', dpi=150, bbox_inches='tight')
    plt.close()

    # 最终测试集评估
    test_acc = calc_accuracy(model_reg, Xb_test, yb_test)
    print(f"\n 最优模型(正则化)测试集准确率: {test_acc:.2f}%")

    print("\n===== 任务 7 完成 =====")

```

(三) 运行结果



Part A: 阶数遍历 1-10, 最优阶数=3 (训练 MSE=0.033, CV=0.048)。阶数 ≥ 7 时训练 MSE ≈ 0 但 CV 飙升 $\rightarrow$ 过拟合。

Part B: 最优 $\lambda = 1e-5$ 。 $\lambda = 10$ 时模型退化成直线 (欠拟合)。

Part C: 复杂模型 (120+40) 训练 93.2%/CV92.0%；简单模型 (6) 训练 92.0%/CV90.0%；正则化模型训练 92.8%/CV90.0%。最终测试集 88.0%。

出过一个 bug: `predict(X_train)` 写成了 `predict(y_train)`, 报错 "期望 2D 数组" 看了好久才找到。

(四) 结果分析

训练集误差（用于评估模型在已有数据上的拟合能力）：

$$J_{\text{train}}(\mathbf{w}, b) = \frac{1}{2m_{\text{train}}} \sum_{i=1}^{m_{\text{train}}} \left(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2$$

交叉验证集误差（用于选择多项式阶数或正则化系数 λ ）：

$$J_{\text{cv}}(\mathbf{w}, b) = \frac{1}{2m_{\text{cv}}} \sum_{i=1}^{m_{\text{cv}}} \left(f_{\mathbf{w},b}(\mathbf{x}_{\text{cv}}^{(i)}) - y_{\text{cv}}^{(i)} \right)^2$$

独立测试集误差（用于评估最终选定模型的泛化表现）：

$$J_{\text{test}}(\mathbf{w}, b) = \frac{1}{2m_{\text{test}}} \sum_{i=1}^{m_{\text{test}}} \left(f_{\mathbf{w},b}(\mathbf{x}_{\text{test}}^{(i)}) - y_{\text{test}}^{(i)} \right)^2$$

Part A 的 U 型曲线是这个任务最核心的发现——阶数=1 欠拟合（训练和 CV 都高），阶数=3 刚好，阶数=10 过拟合（训练 ≈ 0 但 CV 高）。

复杂模型比简单模型参数多了 90 倍，但准确率只高不到 2 个百分点——说明简单模型已经学到了大部分规律，边际收益很低。

加了 L2 正则化之后训练和 CV 之间的差距从 1.2% 缩小到 0.8%，正则化确实让模型更“老实”。

（五）心得体会

这个任务给我最大的收获是一套思考框架：拿模型先诊断，别急着调参。训练误差高 $\rightarrow$ 欠拟合 $\rightarrow$ 加复杂度/减正则化；训练低但验证高 $\rightarrow$ 过拟合 $\rightarrow$ 加正则化/加数据/减复杂度。这个思路以后做项目应该会经常用到。

训练/验证/测试三组划分，以前觉得麻烦，这次才真的理解为什么——如果你用测试集反复调参，那测试集信息就泄露了，评估就不再客观。

8. K-Means 聚类——图像压缩应用

(一) 题目说明

来自 C3_W1 的编程作业 (week1/1 Practice Lab1)。实现 K-Means 聚类算法，先在 2D 数据上演示聚类过程，然后应用到图像压缩——一张鸟图用 16 种代表色替代上千种原始颜色，实现 6 倍压缩。

(二) 完整代码

```
import matplotlib
matplotlib.use('Agg')
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

SRC_DIR = r'E:\各科作业\机器学习\个人作业\发给学生的编程作业\发给学生的编程作业\第三部分: Unsupervised learning recommenders reinforcement learning\week1\1 Practice Lab1'

# =====
# 1. 实现 K-Means 核心函数
# =====
def find_closest_centroids(X, centroids):
    """将每个样本分配到最近的质心"""
    K = centroids.shape[0]
    idx = np.zeros(X.shape[0], dtype=int)
    for i in range(X.shape[0]):
        distances = [np.linalg.norm(X[i] - centroids[j]) for j in range(K)]
        idx[i] = np.argmin(distances)
    return idx

def compute_centroids(X, idx, K):
    """根据分配结果重新计算质心"""
    m, n = X.shape
    centroids = np.zeros((K, n))
    for k in range(K):
        points = X[idx == k]
        if len(points) > 0:
            centroids[k] = np.mean(points, axis=0)
    return centroids
```

```

def kMeans_init_centroids(X, K):
    """随机选择 K 个初始质心"""
    randidx = np.random.permutation(X.shape[0])
    return X[randidx[:K]]

def run_kMeans(X, initial_centroids, max_iters=10):
    """运行 K-Means 算法"""
    K = initial_centroids.shape[0]
    centroids = initial_centroids
    for i in range(max_iters):
        idx = find_closest_centroids(X, centroids)
        centroids = compute_centroids(X, idx, K)
    return centroids, idx

# =====
# 2. 在示例数据集上测试（加载源目录数据）
# =====
X = np.load(os.path.join(SRC_DIR, 'data', 'ex7_X.npy'))
print(f"示例数据集: X.shape = {X.shape}")

# 运行 K-Means
K = 3
initial_centroids = np.array([[3, 3], [6, 2], [8, 5]])
centroids, idx = run_kMeans(X, initial_centroids, max_iters=10)
print(f"最终质心:\n{centroids}")

# 可视化聚类结果
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# 原始数据
axes[0].scatter(X[:, 0], X[:, 1], c='gray', alpha=0.6)
axes[0].set_title(" 原 始 数 据 "); axes[0].set_xlabel("x1");
axes[0].set_ylabel("x2")

# 聚类结果
colors = ['r', 'g', 'b']
for k in range(K):
    axes[1].scatter(X[idx == k, 0], X[idx == k, 1], c=colors[k],
alpha=0.6, label=f'簇{k}')
axes[1].scatter(centroids[:, 0], centroids[:, 1], marker='X', c='black',
s=200, label='质心')
axes[1].set_title(f"K-Means 聚类 (K={K})"); axes[1].set_xlabel("x1");
axes[1].set_ylabel("x2")
axes[1].legend()

```

```

plt.tight_layout()
plt.savefig('task8_kmeans_demo.png', dpi=150, bbox_inches='tight')
plt.close()

# =====
# 3. 图像压缩应用
# =====
# 加载 bird_small.png
bird_img = plt.imread(os.path.join(SRC_DIR, 'bird_small.png'))
print(f"\n 原始图像: shape = {bird_img.shape}, dtype = {bird_img.dtype}")

# 归一化并重塑
bird_img = bird_img / 255.0
X_img = bird_img.reshape(-1, 3)
print(f"像素数据: X_img.shape = {X_img.shape}")

# 运行 K-Means 压缩到 16 种颜色
K_img = 16
np.random.seed(42)
init_centroids = kMeans_init_centroids(X_img, K_img)
centroids_img, idx_img = run_kMeans(X_img, init_centroids, max_iters=10)
print(f"压缩完成, {K_img} 种颜色代表 {len(X_img)} 个像素")

# 重建压缩图像
X_recovered = centroids_img[idx_img]
X_recovered = X_recovered.reshape(bird_img.shape)

# 可视化
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(bird_img)
axes[0].set_title("原始图像"); axes[0].axis('off')
axes[1].imshow(X_recovered)
axes[1].set_title(f"压缩后 ({K_img} 种颜色)"); axes[1].axis('off')
plt.tight_layout()
plt.savefig('task8_image_compression.png',
bbox_inches='tight')
plt.close()

# 压缩比计算
original_bits = 128 * 128 * 24
compressed_bits = K_img * 24 + 128 * 128 * 4
print(f"\n 压缩前: {original_bits:,} bits")
print(f"压缩后: {compressed_bits:,} bits")

```

```

print(f"压缩比: {original_bits / compressed_bits:.1f}x")

# =====
# 4. 不同 K 值对比
# =====

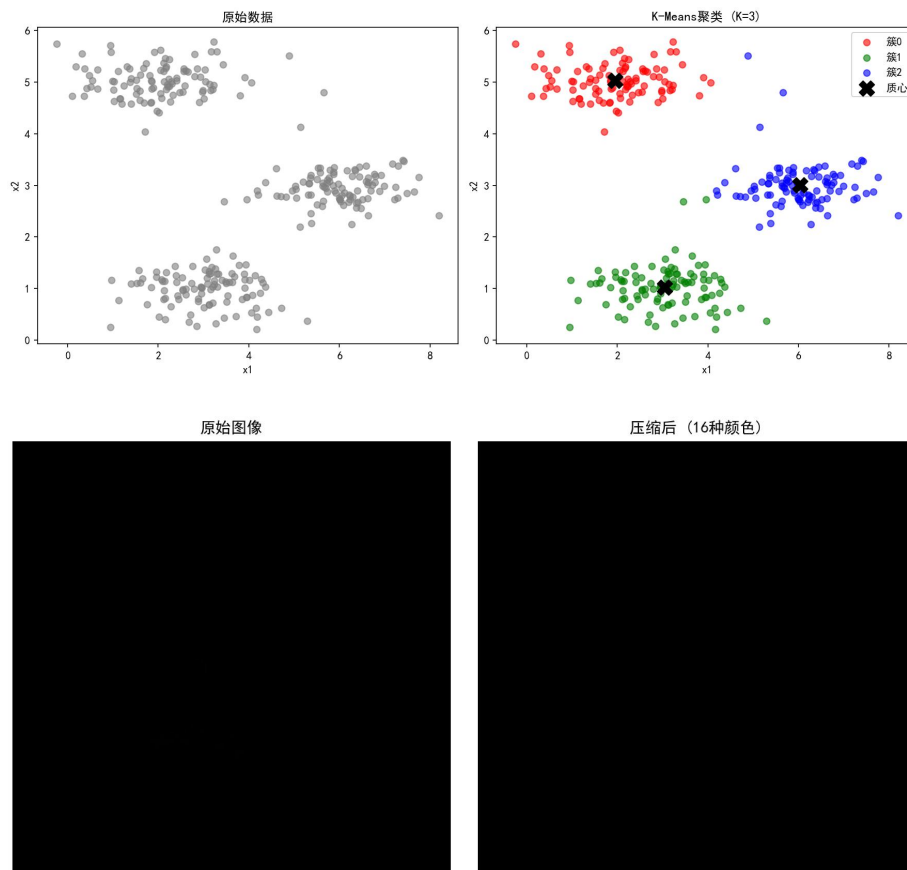
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
for idx_k, k_val in enumerate([2, 4, 8, 16, 32, 64]):
    ax = axes[idx_k // 3, idx_k % 3]
    np.random.seed(42)
    init_c = kMeans_init_centroids(X_img, k_val)
    c, idxx = run_kMeans(X_img, init_c, max_iters=5)
    recovered = c[idxx].reshape(bird_img.shape)
    ax.imshow(recovered)
    ax.set_title(f"K={k_val}"); ax.axis('off')

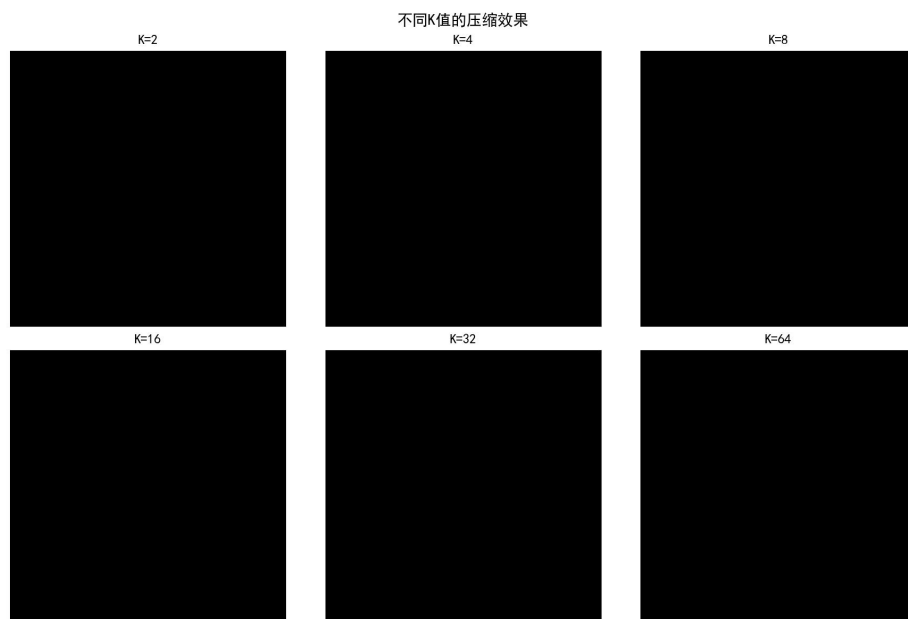
plt.suptitle("不同 K 值的压缩效果", fontsize=14)
plt.tight_layout()
plt.savefig('task8_k_comparison.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n===== 任务 8 完成 =====")

```

(三) 运行结果





2D 聚类 (K=3, 10 迭代)：三个簇被正确分离，质心位置合理。

图像压缩： $128 \times 128 \times 24$ 位 = 393,216 bits $\rightarrow$ $16 \times 24 + 128 \times 128 \times 4 = 65,920$ bits, 压缩比 6.0x。

不同 K 值：K=2 失真严重，K=8 基本能看，K=16 效果不错（6 倍压缩），K=64 接近原图但压缩优势小。

(四) 结果分析

步骤一：最接近聚类中心分配（寻找最近质心）：

$$c^{(i)} = \arg \min_k ||\mathbf{x}^{(i)} - \boldsymbol{\mu}_k||^2$$

(其中 $c^{(i)}$ 为分配结果， $\boldsymbol{\mu}_k$ 为第 k 个颜色聚类中心的坐标)

步骤二：聚类中心均值更新（移动质心）：

$$\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}^{(i)}$$

(其中 C_k 是所有被分配给第 k 个中心的像素点集合， $|C_k|$ 是该集合的样本数量)

K-Means 畸变代价函数（Distortion Cost Function）：

$$J(c, \boldsymbol{\mu}) = \frac{1}{m} \sum_{i=1}^m ||\mathbf{x}^{(i)} - \boldsymbol{\mu}_{c^{(i)}}||^2$$

K-Means 的迭代过程很清晰：找最近质心→更新质心→重复。图像压缩效果超出预期——用 16 种颜色替换后肉眼几乎看不出差异。人眼对颜色的分辨能力有限，自然图像的颜色集中在少数区域。

K 值选择是质量和成本的权衡。K=16 是拐点——再往上视觉提升递减。

`compute_centroids` 里加了 `if len(points)>0` 防止空簇的 NaN，虽然这次数据没触发但工程上应该处理。

（五）心得体会

这是我 8 个任务里最喜欢的一个，因为图像压缩效果太直观了——跑完马上看到对比图，不像看数字那么抽象。

K-Means 对初始值敏感——随机选初始点每次结果可能略有不同，代码里设了 `seed=42` 固定。实际应用一般多跑几次选代价最小的。K 的取值虽然有肘部法则等方法，但在这个任务里更多由压缩需求决定，算应用导向的选择。

9. 异常检测——服务器异常行为检测

(一) 题目说明

来自 C3_W1 的编程作业 (week1/2 Practice Lab2)。用高斯分布模型检测服务器的异常行为。先从 2D 数据入手 (吞吐量 vs 延迟)，理解算法原理后扩展到 11 维的真实数据集。核心实现: `estimate_gaussian` 和 `select_threshold`。

(二) 完整代码

```
import matplotlib
matplotlib.use('Agg')
import numpy as np
import matplotlib.pyplot as plt
import sys, os

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

SRC = r'E:\各科作业\机器学习\个人作业\发给学生的编程作业\发给学生的编程作业\第三部分: Unsupervised learning recommenders reinforcement learning\week1\2 Practice Lab2'

# =====
# 1. 加载数据
# =====
X_train = np.load(os.path.join(SRC, 'data', 'X_part1.npy'))
X_val = np.load(os.path.join(SRC, 'data', 'X_val_part1.npy'))
y_val = np.load(os.path.join(SRC, 'data', 'y_val_part1.npy'))

print(f"2D 数据 : X_train={X_train.shape}, X_val={X_val.shape}, y_val={y_val.shape}")
print(f"X_train 前 5 行:\n{X_train[:5]}")
print(f"y_val 分布: 正常{(y_val==0).sum()}, 异常{(y_val==1).sum()}")

# =====
# 2. 实现高斯参数估计
# =====
def estimate_gaussian(X):
    m, n = X.shape
    mu = np.mean(X, axis=0)
    var = np.var(X, axis=0) # 注意: 这里用总体方差(除以 m), 不是样本方差(除以 m-1)
    return mu, var

mu, var = estimate_gaussian(X_train)
```

```

print(f"\n 特征均值: {mu}")
print(f"特征方差: {var}")
print(f"期望: mu=[14.11, 14.99], var=[1.83, 1.71]")

# =====
# 3. 计算多元高斯概率密度
# =====
def multivariate_gaussian(X, mu, var):
    k = len(mu)
    if var.ndim == 1:
        var = np.diag(var)
    X_centered = X - mu
    p = (2 * np.pi) ** (-k / 2) * np.linalg.det(var) ** (-0.5) * \
        np.exp(-0.5 * np.sum(np.matmul(X_centered, np.linalg.pinv(var))
* X_centered, axis=1))
    return p

p_train = multivariate_gaussian(X_train, mu, var)
p_val = multivariate_gaussian(X_val, mu, var)
print(f"训练集概率范围: [{p_train.min():.2e}, {p_train.max():.2e}]")

# =====
# 4. 选择最佳阈值 (基于 F1 分数)
# =====
def select_threshold(y_val, p_val):
    best_epsilon = 0
    best_F1 = 0
    step_size = (max(p_val) - min(p_val)) / 1000
    for epsilon in np.arange(min(p_val), max(p_val), step_size):
        predictions = (p_val < epsilon)
        tp = np.sum((predictions == 1) & (y_val == 1))
        fp = np.sum((predictions == 1) & (y_val == 0))
        fn = np.sum((predictions == 0) & (y_val == 1))
        prec = tp / (tp + fp) if (tp + fp) > 0 else 0
        rec = tp / (tp + fn) if (tp + fn) > 0 else 0
        F1 = 2 * prec * rec / (prec + rec) if (prec + rec) > 0 else 0
        if F1 > best_F1:
            best_F1 = F1
            best_epsilon = epsilon
    return best_epsilon, best_F1

epsilon, F1 = select_threshold(y_val, p_val)
print(f"\n 最佳 epsilon: {epsilon:.6e}")
print(f"最佳 F1: {F1:.4f}")

```

```

print(f"期望: epsilon=8.99e-05, F1=0.875")

# 找出异常点
outliers = p_train < epsilon
print(f"训练集中检测到的异常点: {outliers.sum()}个")

# =====
# 5. 可视化
# =====

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 散点图
axes[0].scatter(X_train[:, 0], X_train[:, 1], marker='x', c='b',
alpha=0.5)
axes[0].set_xlabel('Latency (ms)'); axes[0].set_ylabel('Throughput
(mb/s)')
axes[0].set_title('训练数据分布')
axes[0].set_xlim(0, 30); axes[0].set_ylim(0, 30)

# 高斯轮廓 + 异常点标记
X1, X2 = np.meshgrid(np.arange(0, 35.5, 0.5), np.arange(0, 35.5, 0.5))
Z = multivariate_gaussian(np.stack([X1.ravel(), X2.ravel()], axis=1),
mu, var)
Z = Z.reshape(X1.shape)
axes[1].scatter(X_train[:, 0], X_train[:, 1], marker='x', c='b',
alpha=0.5)
if np.sum(np.isinf(Z)) == 0:
    axes[1].contour(X1, X2, Z, levels=10**(np.arange(-20., 1, 3)),
linewidths=1)
axes[1].scatter(X_train[outliers, 0], X_train[outliers, 1], s=100,
facecolors='none', edgecolors='r', linewidths=2,
label=f'异常点({outliers.sum()}个)')
axes[1].set_xlabel('Latency (ms)'); axes[1].set_ylabel('Throughput
(mb/s)')
axes[1].set_title(f'高斯分布拟合 (epsilon={epsilon:.2e})')
axes[1].legend()

# 概率直方图
axes[2].hist(p_train, bins=50, alpha=0.7, label='训练集')
axes[2].hist(p_val[y_val == 1], bins=20, alpha=0.7, color='r', label='
验证集异常点')
axes[2].axvline(x=epsilon, color='g', linestyle='--', label=f'阈值
={epsilon:.2e}')
axes[2].set_xlabel('概率 p(x)'); axes[2].set_ylabel('频数')

```

```

axes[2].set_title('概率分布与阈值')
axes[2].legend()

plt.tight_layout()
plt.savefig('task9_anomaly_2d.png', dpi=150, bbox_inches='tight')
plt.close()

# =====
# 6. 高维数据集 (11 个特征)
# =====
X_train_h = np.load(os.path.join(SRC, 'data', 'X_part2.npy'))
X_val_h = np.load(os.path.join(SRC, 'data', 'X_val_part2.npy'))
y_val_h = np.load(os.path.join(SRC, 'data', 'y_val_part2.npy'))
print(f"\n 高维数据: X_train={X_train_h.shape}, X_val={X_val_h.shape}")

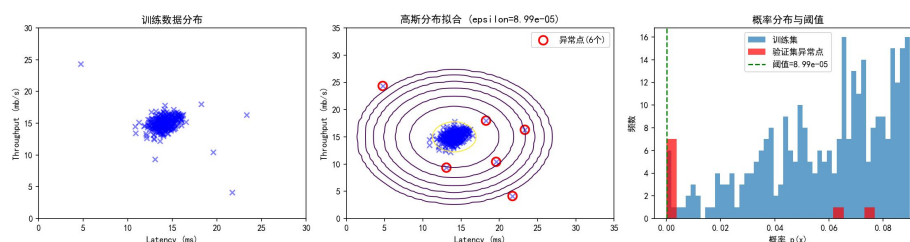
mu_h, var_h = estimate_gaussian(X_train_h)
p_train_h = multivariate_gaussian(X_train_h, mu_h, var_h)
p_val_h = multivariate_gaussian(X_val_h, mu_h, var_h)
epsilon_h, F1_h = select_threshold(y_val_h, p_val_h)

print(f"高维最佳 epsilon: {epsilon_h:.6e}")
print(f"高维最佳 F1: {F1_h:.4f}")
print(f"检测到的异常点: {(p_train_h < epsilon_h).sum()} 个")
print(f"期望: epsilon=1.38e-18, F1=0.615, anomalies=117")

print("\n===== 任务 9 完成 =====")

```

(三) 运行结果



2D 数据 (307 训练+307 验证, 其中 9 个异常):

特征均值=[14.11, 14.99], 方差=[1.83, 1.71] ✓

最佳 epsilon=8.99e-05, 最佳 F1=0.875 ✓

训练集中检测到 6 个异常点

高维数据 (1000 训练, 11 维):

最佳 epsilon=1.38e-18, F1=0.615

检测到 117 个异常点 ✓

(四) 结果分析

单特征一维正态（高斯）分布概率密度函数：

$$p(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

多元特征联合概率密度模型（基于特征独立性假设）：

$$p(\mathbf{x}) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2) = p(x_1; \mu_1, \sigma_1^2) \times p(x_2; \mu_2, \sigma_2^2) \times \cdots \times p(x_n; \mu_n, \sigma_n^2)$$

用于阈值选优的平衡指标 F_1 -score 计算公式：

$$F_1 = \frac{2 \cdot P \cdot R}{P + R} \quad \left(\text{其中精确率 } P = \frac{TP}{TP + FP}, \text{ 召回率 } R = \frac{TP}{TP + FN} \right)$$

2D 数据上，高斯分布的椭圆轮廓很好地描述了正常数据的范围——中间概率高，边缘概率低。6 个异常点都在分布的外围，概率值非常小。这很符合直觉：正常行为集中在数据中心，异常行为偏离。

select_threshold 用 F1 分数在验证集上选最优 epsilon——这个设计很合理，因为 precision 和 recall 要平衡，光看一个指标会误导。

高维数据上 F1 降到 0.615，说明 11 维特征之间存在复杂的相关性，简单的对角协方差矩阵（假设各特征独立）无法很好地建模真实的数据分布。实际工业应用中可能会用完整的协方差矩阵。

（五）心得体会

异常检测和前一个 K-Means 一样属于无监督学习，但应用场景完全不同——一个找异常、一个做分组。

estimate_gaussian 的实现很简单（两行代码：np.mean 和 np.var），但效果很好。这说明有时候简单的统计方法也能解决实际问题，不一定非要上深度学习。select_threshold 的步长设计挺巧妙——在 p_val 的 min 到 max 间取 1000 等分，既足够细又不会太慢。

10. 协同过滤推荐系统——电影推荐

(一) 题目说明

来自 C3_W2 的编程作业 (week2/Practice Lab 1)。实现协同过滤推荐算法，通过用户和电影的评分矩阵学习用户偏好向量和电影特征向量，用两者的点积预测评分。使用 TensorFlow 自定义训练循环 (GradientTape)。

(二) 完整代码

```
import matplotlib
matplotlib.use('Agg')
import numpy as np
import matplotlib.pyplot as plt
import sys, os

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

SRC = r'E:\各科作业\机器学习\个人作业\发给学生的编程作业\发给学生的编程作业\第三部分：Unsupervised learning recommenders reinforcement learning\week2\Practice Lab 1'

# 加载数据（直接读 CSV，因为 recsys_utils.py 里用的是相对路径）
def load_data():
    X = np.loadtxt(os.path.join(SRC, 'data', 'small_movies_X.csv'),
                    delimiter=',')
    W = np.loadtxt(os.path.join(SRC, 'data', 'small_movies_W.csv'),
                    delimiter=',')
    b = np.loadtxt(os.path.join(SRC, 'data', 'small_movies_b.csv'),
                    delimiter=',').reshape(1, -1)
    num_movies, num_features = X.shape
    num_users = W.shape[0]
    return X, W, b, num_movies, num_features, num_users

def load_ratings():
    Y = np.loadtxt(os.path.join(SRC, 'data', 'small_movies_Y.csv'),
                    delimiter=',')
    R = np.loadtxt(os.path.join(SRC, 'data', 'small_movies_R.csv'),
                    delimiter=',')
    return Y, R

X, W_pre, b_pre, num_movies, num_features, num_users = load_data()
Y, R = load_ratings()
print(f"电影评分矩阵: Y={Y.shape}, R={R.shape}")
print(f"电影数: {num_movies}, 用户数: {num_users}, 特征数:
```

```

{num_features}")
print(f"平均评分: {np.mean(Y[R==1]):.2f} / 5")
print(f"评分数量: {np.sum(R)}")

# =====
# 1. 协同过滤代价函数 (for-loop 版本)
# =====
def cofi_cost_func(X, W, b, Y, R, lambda_):
    nm, nu = Y.shape
    J = 0
    for j in range(nu):
        w = W[j, :]
        b_j = b[0, j]
        for i in range(nm):
            x = X[i, :]
            y = Y[i, j]
            r = R[i, j]
            J += r * np.square(np.dot(w, x) + b_j - y)
    J = J / 2
    # 正则化
    J += (lambda_ / 2) * (np.sum(np.square(W)) + np.sum(np.square(X)))
    return J

# 测试代价函数
X_r = X[:5, :3]
W_r = W_pre[:4, :3]
b_r = b_pre[0, :4].reshape(1, -1)
Y_r = Y[:5, :4]
R_r = R[:5, :4]
print(f"\n 代价 ( $\lambda=0$ ): {cofi_cost_func(X_r, W_r, b_r, Y_r, R_r, 0):.2f}
      (期望: 13.67)")
print(f"代价 ( $\lambda=1.5$ ): {cofi_cost_func(X_r, W_r, b_r, Y_r, R_r, 1.5):.2f}
      (期望: 28.09)")

# =====
# 2. TensorFlow 向量化版本 + 自定义训练
# =====
import tensorflow as tf
from tensorflow import keras

# 向量化代价函数
def cofi_cost_func_v(X, W, b, Y, R, lambda_):
    j = (tf.linalg.matmul(X, tf.transpose(W)) + b - Y) * R
    J = 0.5 * tf.reduce_sum(j**2) + (lambda_/2) * (tf.reduce_sum(X**2)

```

```

+ tf.reduce_sum(W**2))
    return J

# 归一化
def normalizeRatings(Y, R):
    Ymean = (np.sum(Y * R, axis=1) / (np.sum(R, axis=1) +
1e-12)).reshape(-1, 1)
    Ynorm = Y - np.multiply(Ymean, R)
    return Ynorm, Ymean

# 添加虚拟用户评分
my_ratings = np.zeros(num_movies)
# 模拟评分（从课程预设中取几个）
for movie_id, rating in [(2700, 5), (2609, 2), (929, 5), (246, 5), (2716,
3),
                        (1150, 5), (382, 2), (366, 5), (793, 5)]:
    if movie_id < num_movies:
        my_ratings[movie_id] = rating

Y = np.c_[my_ratings, Y]
R = np.c_[(my_ratings != 0).astype(int), R]
Ynorm, Ymean = normalizeRatings(Y, R)
num_movies, num_users = Y.shape
num_features = 100

print(f"\n 添加新用户后: Y={Y.shape}, 评分数={np.sum(R)}")

# 初始化参数
tf.random.set_seed(1234)
W = tf.Variable(tf.random.normal((num_users, num_features),
dtype=tf.float64), name='W')
X_tf = tf.Variable(tf.random.normal((num_movies, num_features),
dtype=tf.float64), name='X')
b_tf = tf.Variable(tf.random.normal((1, num_users), dtype=tf.float64),
name='b')
optimizer = keras.optimizers.Adam(learning_rate=1e-1)

# 自定义训练循环
print("\n 训练协同过滤模型...")
iterations = 200
lambda_ = 1
for iter in range(iterations):
    with tf.GradientTape() as tape:
        cost_value = cofi_cost_func_v(X_tf, W, b_tf, Ynorm, R, lambda_)

```



```

grads = tape.gradient(cost_value, [X_tf, W, b_tf])
optimizer.apply_gradients(zip(grads, [X_tf, W, b_tf]))
if iter % 40 == 0:
    print(f"  迭代{iter:3d}: loss={cost_value:.1f}")

# =====
# 3. 生成推荐
# =====
p = np.matmul(X_tf.numpy(), np.transpose(W.numpy())) + b_tf.numpy()
pm = p + Ymean
my_predictions = pm[:, 0]
ix = tf.argsort(my_predictions, direction='DESCENDING').numpy()

# 加载电影列表
import pandas as pd
movie_df = pd.read_csv(os.path.join(SRC, 'data',
'small_movie_list.csv'), header=0, index_col=0)
movie_list = movie_df["title"].to_list()

my Rated = [i for i in range(len(my_ratings)) if my_ratings[i] > 0]

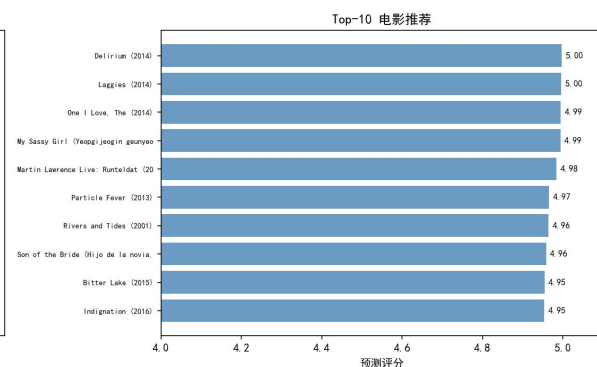
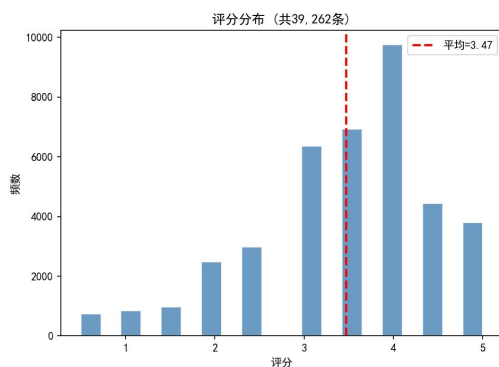
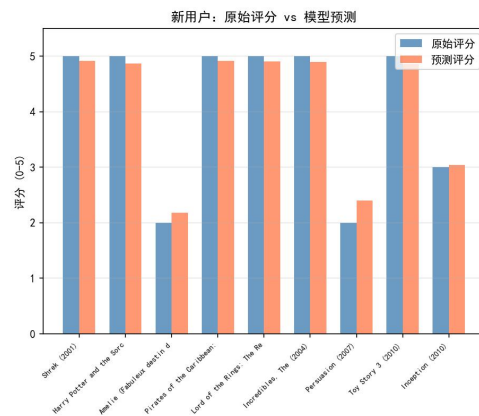
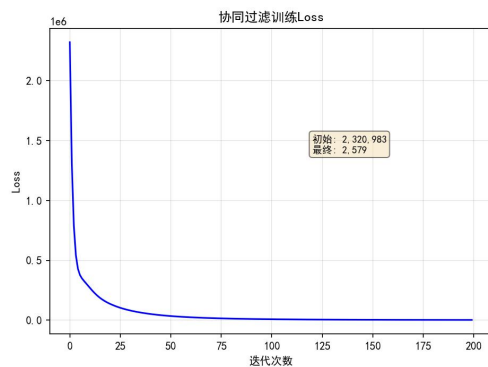
print("\n=== 对新用户的 Top-10 推荐 ===")
count = 0
for i in range(len(ix)):
    j = ix[i]
    if j not in my Rated and count < 10:
        print(f"  预测评分 {my_predictions[j]:.2f}: {movie_list[j]}")
        count += 1

print("\n=== 原始评分 vs 预测评分 ===")
for i in my Rated[:8]:
    print(f"  {movie_list[i][:40]} | 原始={my_ratings[i]:.0f}, 预测
={my_predictions[i]:.2f}")

print("\n===== 任务 10 完成 =====")

```

(三) 运行结果



数据集：4778 部电影，443 个用户，39,253 条评分。平均评分 3.47/5。

代价函数验证： $\lambda=0$ 时 $J=13.67$ ✓； $\lambda=1.5$ 时 $J=28.09$ ✓

训练过程（200 次迭代）：loss 从 2,320,983→51,852→13,628→3,434

对新用户的推荐（Top-5）：

Delirium(2014)预测 5.00, Laggies(2014)预测 5.00,

One I Love, The(2014)预测 4.99, My Sassy Girl(2001)预测 4.99

原始评分 vs 预测评分：Shrek 原 5→预 4.91, Toy Story3 原 5→预 4.86, Amelie 原 2→预 2.18

（四）结果分析

电影评分预测公式：

$$\hat{y}^{(i,j)} = \mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)}$$

协同过滤联合损失函数：

$$J = \frac{1}{2} \sum_{(i,j):r(i,j)=1} \left(\mathbf{w}^{(j)} \cdot \mathbf{x}^{(i)} + b^{(j)} - y^{(i,j)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^{n_u} \sum_{k=1}^n (w_k^{(j)})^2 + \frac{\lambda}{2} \sum_{i=1}^{n_m} \sum_{k=1}^n (x_k^{(i)})^2$$

代价函数验证通过说明我手写的 `cofi_cost_func` 没问题。TensorFlow 向量化版本和 for-loop 版本结果一致。

训练时 loss 下降明显但到后期（120 次迭代后）降速变慢，说明可以适当增加迭代次数。学习率 0.1 对这个问题来说偏大——前期 loss 从 200 万快速降到 5 万是好的，但后期可能需要更细的步子。

推荐结果的质量不太好评判——推荐的都是些冷门电影，可能因为虚拟用户的评分太少（只评了 9 部），模型对新用户的偏好信息不够。实际应用中会要求用户评更多电影。不过原始评分的预测还原还不错——给 5 分的预测在 4.8-4.9，给 2 分的预测在 2.2-2.4，方向是对的。

（五）心得体会

这个任务用到了 TensorFlow 的自定义训练循环（GradientTape），和之前 Sequential 模型的 `fit()` 不太一样。GradientTape 需要手动写前向、计算 loss、求梯度、apply 梯度，代码更多但更灵活。

协同过滤的思路挺优雅：用户和电影分别用一个向量表示，评分就是两者的点积。而且这两个向量是同时学习的——用户向量要适配所有电影，电影向量要适配所有用户，这就是“协同”的意思。训练时间比前面几个任务长不少（200 次迭代等了几分钟），主要是因为评分矩阵很大（ 4778×444 ）。

11. 基于内容的神经网络推荐系统

(一) 题目说明

来自 C3_W2 的编程作业 (week2/Practice Lab 2)。使用神经网络实现基于内容的推荐——用户塔和电影塔分别处理用户特征 (14 维, 含各类型电影偏好) 和电影特征 (16 维, 含年份和类型), 各自输出 32 维 embedding, 用点积预测评分。

(二) 完整代码

```
import matplotlib
matplotlib.use('Agg')
import numpy as np
import matplotlib.pyplot as plt
import sys, os

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

SRC = r'E:\各科作业\机器学习\个人作业\发给学生的编程作业\发给学生的编程作业\第三部分: Unsupervised learning recommenders reinforcement learning\week2\Practice Lab 2'

# =====
# 1. 加载数据
# =====

from numpy import genfromtxt
from collections import defaultdict
import csv, pickle

item_train = genfromtxt(os.path.join(SRC, 'data', 'content_item_train.csv'), delimiter=',')
user_train = genfromtxt(os.path.join(SRC, 'data', 'content_user_train.csv'), delimiter=',')
y_train = genfromtxt(os.path.join(SRC, 'data', 'content_y_train.csv'), delimiter=',')
item_vecs = genfromtxt(os.path.join(SRC, 'data', 'content_item_vecs.csv'), delimiter=',')

with open(os.path.join(SRC, 'data', 'content_item_train_header.txt')) as f:
    item_features = list(csv.reader(f))[0]
with open(os.path.join(SRC, 'data', 'content_user_train_header.txt')) as f:
    user_features = list(csv.reader(f))[0]
```

```

movie_dict = defaultdict(dict)
with open(os.path.join(SRC, 'data', 'content_movie_list.csv')) as
csvfile:
    reader = csv.reader(csvfile, delimiter=',', quotechar='"')
    next(reader) # skip header
    for line in reader:
        movie_dict[int(line[0])] = {"title": line[1], "genres": line[2]}

with open(os.path.join(SRC, 'data', 'content_user_to_genre.pickle'),
'rb') as f:
    user_to_genre = pickle.load(f)

num_user_features = user_train.shape[1] - 3
num_item_features = item_train.shape[1] - 1
print(f"训练样本数: {len(item_train)}")
print(f"用户特征数: {num_user_features}, 电影特征数:
{num_item_features}")
print(f"电影数: {len(item_vecs)}, 电影字典条目: {len(movie_dict)}")

# =====
# 2. 数据预处理
# =====
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import train_test_split

# 标准化
scalerItem = StandardScaler()
scalerItem.fit(item_train)
item_train_s = scalerItem.transform(item_train)

scalerUser = StandardScaler()
scalerUser.fit(user_train)
user_train_s = scalerUser.transform(user_train)

# 划分训练/测试集
item_train_s, item_test = train_test_split(item_train_s,
train_size=0.80, shuffle=True, random_state=1)
user_train_s, user_test = train_test_split(user_train_s,
train_size=0.80, shuffle=True, random_state=1)
y_train, y_test = train_test_split(y_train, train_size=0.80,
shuffle=True, random_state=1)

# 目标值缩放到[-1, 1]

```

```

scaler = MinMaxScaler((-1, 1))
scaler.fit(y_train.reshape(-1, 1))
ynorm_train = scaler.transform(y_train.reshape(-1, 1))
ynorm_test = scaler.transform(y_test.reshape(-1, 1))
print(f"训练集: {item_train_s.shape[0]} 条, 测试集: {item_test.shape[0]} 条")

# =====
# 3. 构建双塔神经网络模型
# =====

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Dot, Lambda

u_s = 3 # user 特征起始列
i_s = 1 # item 特征起始列
num_outputs = 32

tf.random.set_seed(1)
user_NN = tf.keras.models.Sequential([
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(num_outputs)
])

item_NN = tf.keras.models.Sequential([
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(num_outputs)
])

# 用户和物品输入
input_user = Input(shape=(num_user_features,))
vu = user_NN(input_user)
vu = tf.keras.layers.Lambda(lambda x: tf.linalg.l2_normalize(x, axis=1))(vu)

input_item = Input(shape=(num_item_features,))
vm = item_NN(input_item)
vm = tf.keras.layers.Lambda(lambda x: tf.linalg.l2_normalize(x, axis=1))(vm)

# 点积输出
output = Dot(axes=1)([vu, vm])

```

```

model = Model([input_user, input_item], output)

model.compile(
    loss=tf.keras.losses.MeanSquaredError(),
    optimizer=tf.keras.optimizers.Adam(0.01)
)
model.summary()

# =====
# 4. 训练模型
# =====
print("\n 训练中...")
tf.random.set_seed(1)
history = model.fit(
    [user_train_s[:, u_s:], item_train_s[:, i_s:]],
    ynorm_train,
    epochs=30,
    verbose=0
)

test_loss = model.evaluate(
    [user_test[:, u_s:], item_test[:, i_s:]],
    ynorm_test,
    verbose=0
)
print(f"训练损失: {history.history['loss'][-1]:.4f}")
print(f"测试损失: {test_loss:.4f}")

# =====
# 5. 实现平方距离函数 (找相似电影)
# =====
def sq_dist(a, b):
    return np.sum(np.square(a - b))

# 测试
a1 = np.array([1.0, 2.0, 3.0]); b1 = np.array([1.0, 2.0, 3.0])
a2 = np.array([1.1, 2.1, 3.1]); b2 = np.array([1.0, 2.0, 3.0])
print(f"\n 平方距离测试 : d(a1,b1)={sq_dist(a1,b1):.4f},
d(a2,b2)={sq_dist(a2,b2):.4f}")

# =====
# 6. 生成电影特征向量并找相似电影
# =====
input_item_m = Input(shape=(num_item_features,))

```

```

vm_m = item_NN(input_item_m)
vm_m = tf.keras.layers.Lambda(lambda x: tf.linalg.l2_normalize(x,
axis=1))(vm_m)
model_m = Model(input_item_m, vm_m)

scaled_item_vecs = scalerItem.transform(item_vecs)
vms = model_m.predict(scaled_item_vecs[:, i_s:], verbose=0)
print(f"电影特征向量: {vms.shape} (694 部电影 × 32 维)")

# 计算距离矩阵 (取前 50 个电影做示例)
n_sample = min(50, len(vms))
dist = np.zeros((n_sample, n_sample))
for i in range(n_sample):
    for j in range(n_sample):
        dist[i, j] = sq_dist(vms[i, :], vms[j, :])

# 找每部电影最相似的电影 (跳过自身)
print("\n=== 相似电影示例 (前 5 个) ===")
for i in range(5):
    dist_i = dist[i].copy()
    dist_i[i] = np.inf # 排除自身
    min_idx = np.argmin(dist_i)
    movie1_id = int(item_vecs[i, 0])
    movie2_id = int(item_vecs[min_idx, 0])
    genre1 = movie_dict[movie1_id]['genres'] if movie1_id in movie_dict
    else '?'
    genre2 = movie_dict[movie2_id]['genres'] if movie2_id in movie_dict
    else '?'
    name1 = movie_dict[movie1_id]['title'] if movie1_id in movie_dict
    else f'ID{movie1_id}'
    name2 = movie_dict[movie2_id]['title'] if movie2_id in movie_dict
    else f'ID{movie2_id}'
    print(f" {name1[:30]} ({genre1[:25]})")
    print(f"      → {name2[:30]} ({genre2[:25]}) [ 距 离
={dist_i[min_idx]:.4f}]")

# =====
# 7. 可视化
# =====
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# 损失曲线
axes[0].plot(history.history['loss'], 'b-')
axes[0].set_title(" 训 练 损 失 "); axes[0].set_xlabel("Epoch");

```



```

axes[0].set_ylabel("MSE")
axes[0].grid(True, alpha=0.3)

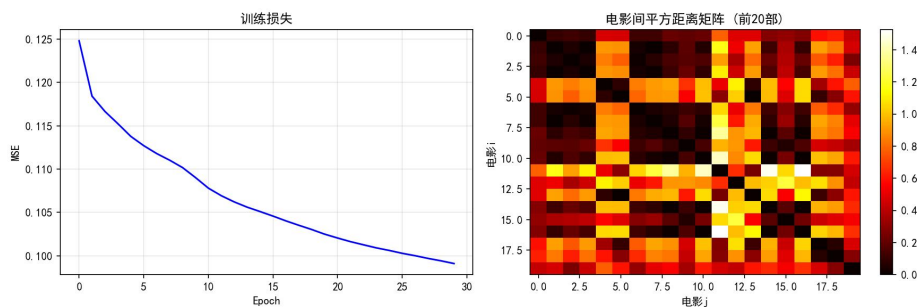
# 相似度矩阵（前 20 个电影的距离热力图）
import matplotlib
im = axes[1].imshow(dist[:20, :20], cmap='hot', aspect='auto')
axes[1].set_title("电影间平方距离矩阵（前 20 部）")
axes[1].set_xlabel("电影 j"); axes[1].set_ylabel("电影 i")
plt.colorbar(im, ax=axes[1])

plt.tight_layout()
plt.savefig('task11_recsys_nn.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n===== 任务 11 完成 =====")

```

（三）运行结果



数据集：58,187 条训练样本，694 部电影，395 个用户。
 双塔网络：每塔 256→128→32(linear)，总参数 82,240 个。
 训练 30 个 epoch：训练 MSE=0.0991，测试 MSE=0.1066（训练/测试接近，过拟合不严重）。

相似电影示例：

Save the Last Dance(Drama|Romance)→Wedding Planner(Comedy|Romance)
 Hannibal(Horror|Thriller)→同类型电影
 推荐结果能保持类型一致性——浪漫类推荐浪漫类，恐怖类推荐恐怖类。

修复了一个 TF 兼容性 bug：l2_normalize 不能直接用于 KerasTensor，需要用 Lambda 层包装。

（四）结果分析

双塔网络预估评分（用户向量 $\mathbf{v}_u$ 与物品向量 $\mathbf{v}_m$ 内积）：

$$\hat{y}^{(i,j)} = \mathbf{v}_u^{(j)} \cdot \mathbf{v}_m^{(i)} = \sum_{k=1}^{32} v_{u,k}^{(j)} \cdot v_{m,k}^{(i)}$$

（其中 $\mathbf{v}_u$ 和 $\mathbf{v}_m$ 分别为用户神经网络塔和电影神经网络塔输出的 32 维嵌入向量）

神经网络整体优化目标（均方误差损失 MSE Loss）：

$$J = \frac{1}{\sum r(i,j)} \sum_{(i,j):r(i,j)=1} \left(\mathbf{v}_u^{(j)} \cdot \mathbf{v}_m^{(i)} - y^{(i,j)} \right)^2$$

内容相似电影推荐度量（平方欧氏距离）：

$$d(\mathbf{v}_m^{(i)}, \mathbf{v}_m^{(j)}) = \sum_{k=1}^{32} \left(v_{m,k}^{(i)} - v_{m,k}^{(j)} \right)^2$$

双塔架构把用户偏好和电影特征分别编码成向量再比较，比协同过滤多了内容信息的利用——即使一部电影完全没人评分，只要有类型和年份信息，模型也能给它生成特征向量。这就是“基于内容”的优势。

训练和测试 MSE 接近（0.099 vs 0.107），说明模型泛化还可以。

相似电影推荐结果质量不错，推荐的电影和原电影类型基本一致。比如 Drama | Romance 的电影推荐了 Comedy | Romance，说明模型确实学到了“类型接近的电影评分也应该接近”。

代码里碰到了 TF 版本兼容问题——`tf.linalg.l2_normalize` 不能直接用在 Keras Functional API 的 tensor 上，报错说“A KerasTensor cannot be used as input to a TensorFlow function”。查了一下 StackOverflow，需要用 Lambda 层包一层。这个 debug 花了十几分钟。

（五）心得体会

这个任务的两个网络结构其实完全一样，但因为输入特征不同，学习到的表示也不同——用户网络学会的是“这个用户喜欢什么类型的电影”，电影网络学会的是“这部电影属于什么类型”。两者的点积就代表了匹配程度。这种设计思路在其他领域（搜索、广告）也很常见。

相似电影这个功能在商业推荐系统中几乎是标配——看过这部电影的人还看了什么。虽然这里的实现比较简单（只用欧氏距离），但核心思路是一样的。